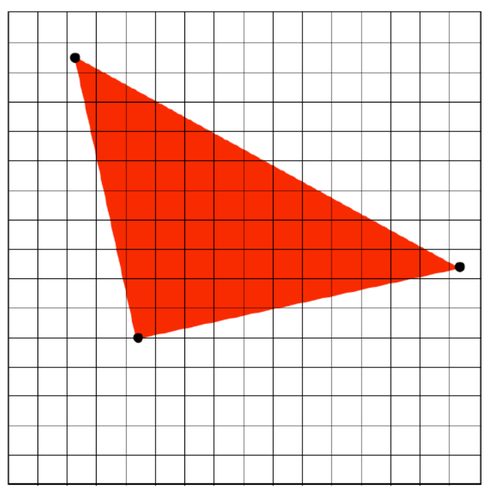
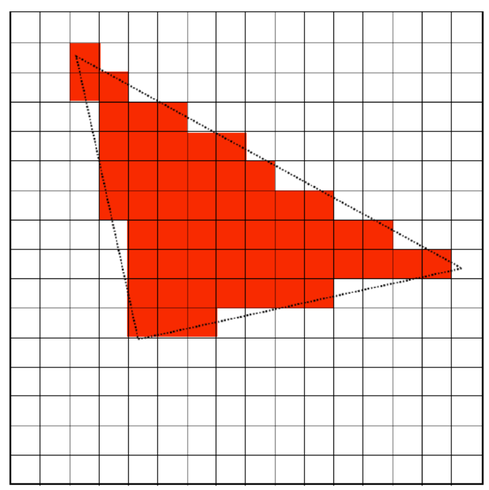
Drawing a Triangle (and an Intro to Sampling)

TODAY: Rasterization

- Two major techniques for "getting stuff on the screen"
- Rasterization (TODAY)
  - *for each primitive* (e.g., triangle), which pixels light up?
  - extremely fast (BILLIONS of triangles per second on GPU)
  - harder (but not impossible) to achieve photorealism
  - perfect match for 2D vector art, fonts, quick 3D preview, . . .
- Ray tracing (LATER)
  - *for each pixel,* which primitives are seen?
  - easier to get photorealism
  - generally slower
  - much more later in the semester!

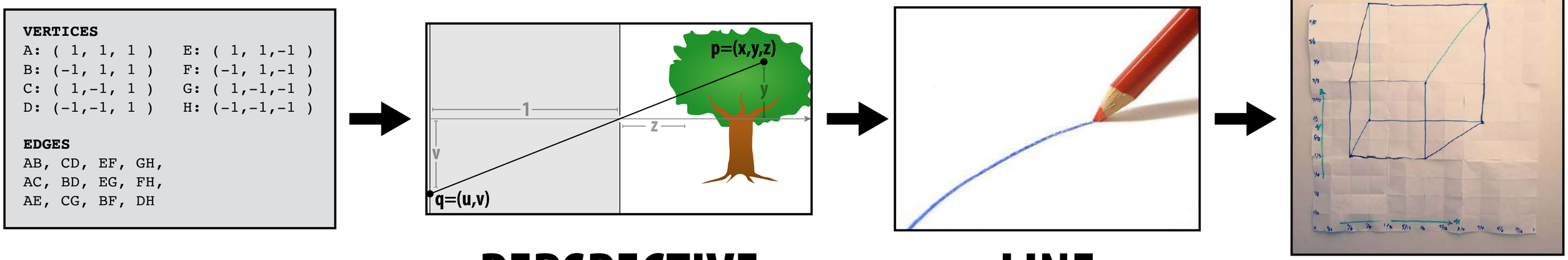




3D Image Generation Pipeline(s)

- Can talk about image generation in terms of a "pipeline":
  - INPUTS — what image do we want to draw?
  - STAGES — sequence of transformations from input → output
  - OUTPUTS — the final image

E.g., our pipeline from the first lecture:



INPUT

PERSPECTIVE PROJECTION STAGE

LINE DRAWING STAGE

OUTPUT

Rasterization Pipeline

- Modern real time image generation based on *rasterization*
  - INPUT: 3D "primitives"—essentially all triangles!
    - possibly with additional attributes (e.g., color)
  - OUTPUT: bitmap image (possibly w/ depth, alpha, . . .)
- Our goal: understand the stages in between*

INPUT (TRIANGLES)

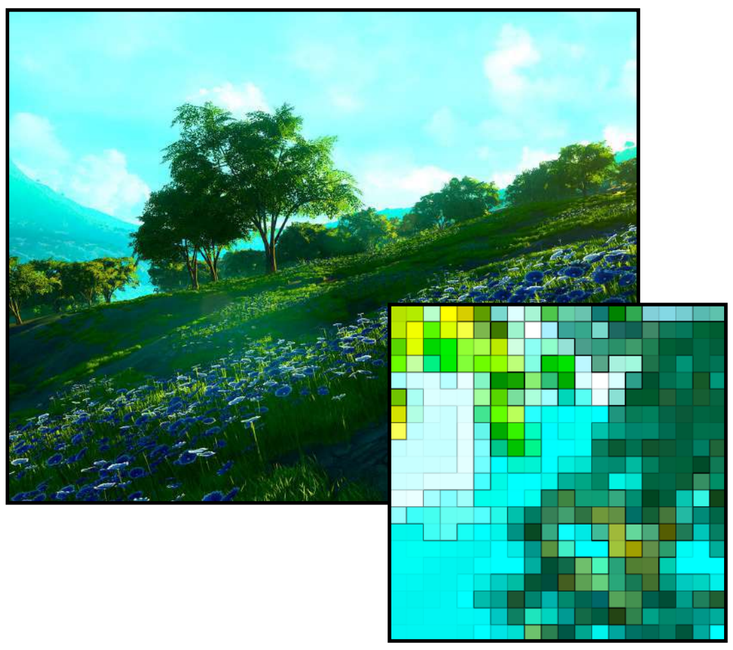
```
VERTICES
A: ( 1, 1, 1 )   E: ( 1, 1,-1 )
B: (-1, 1, 1 )   F: (-1, 1,-1 )
C: ( 1,-1, 1 )   G: ( 1,-1,-1 )
D: (-1,-1, 1 )   H: (-1,-1,-1 )

TRIANGLES
EHF, GFH, FGB, CBG,
GHC, DCH, ABD, CDB,
HED, ADE, EFA, BAF
```

RASTERIZATION PIPELINE



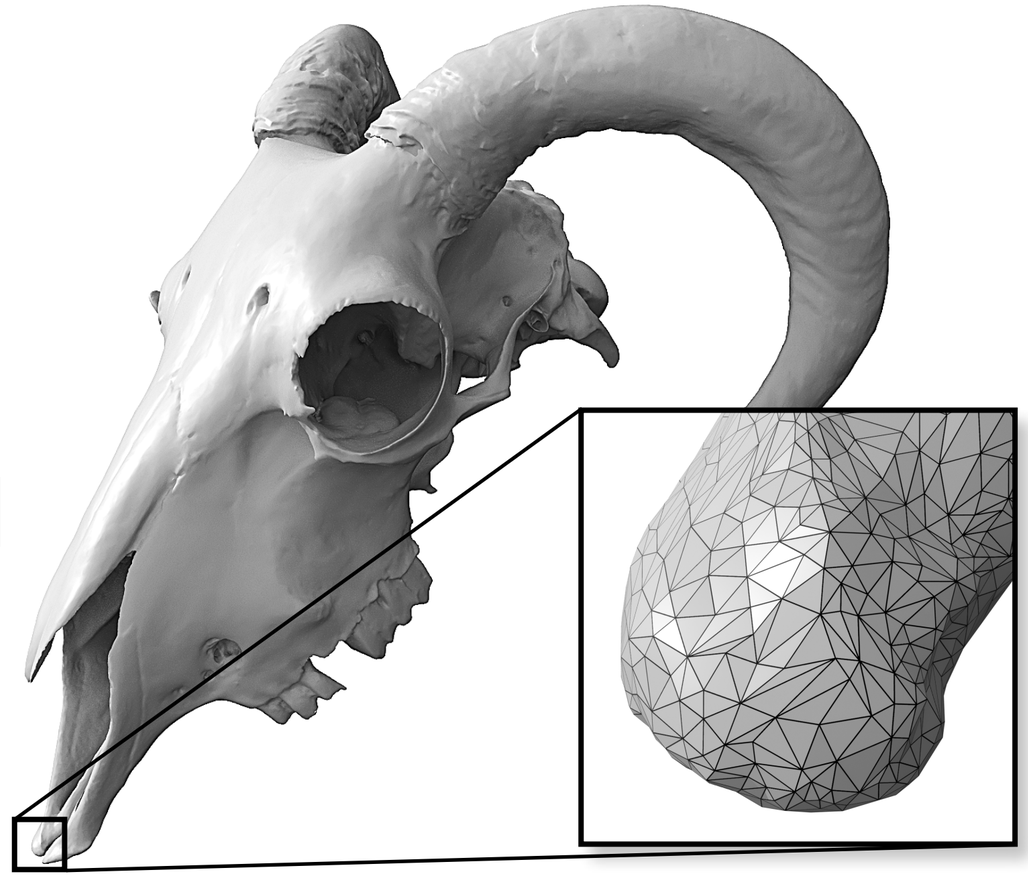
OUTPUT (BITMAP IMAGE)



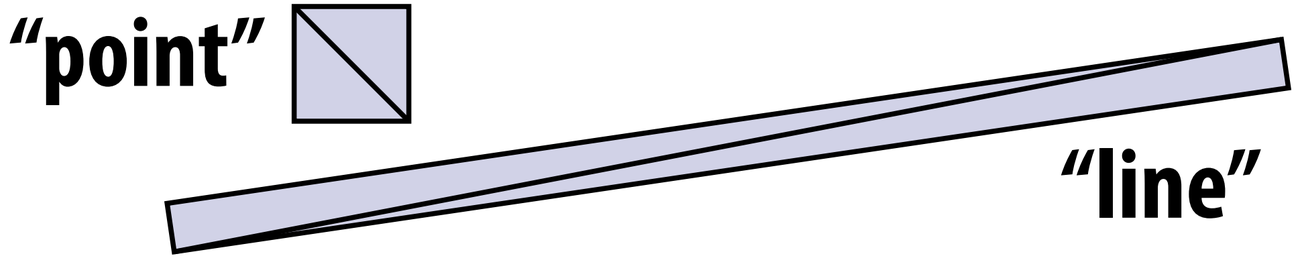
*In practice, usually executed by *graphics processing unit (GPU)*

Why triangles?

- Rasterization pipeline converts <u>all</u> primitives to triangles
  - even points and lines!
- Why?
  - can approximate any shape
  - always planar, well-defined normal
  - easy to interpolate data at corners
    - *"barycentric coordinates"*

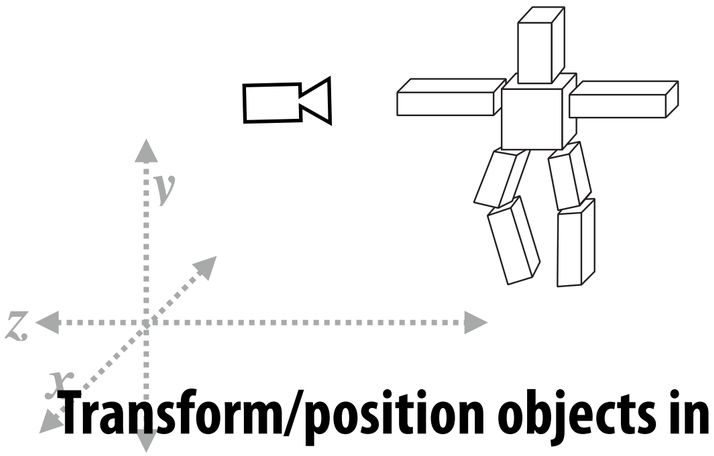


- *Key reason*: once everything is reduced to triangles, can focus on making an extremely well-optimized <u>pipeline</u> for drawing them

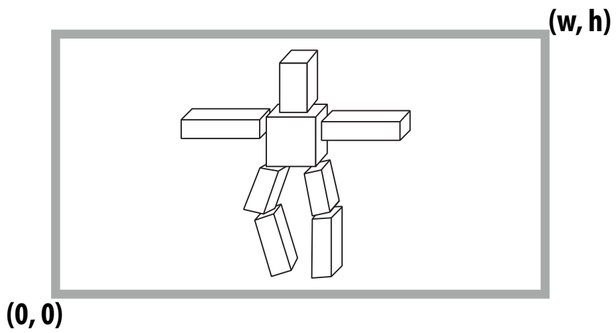


The Rasterization Pipeline

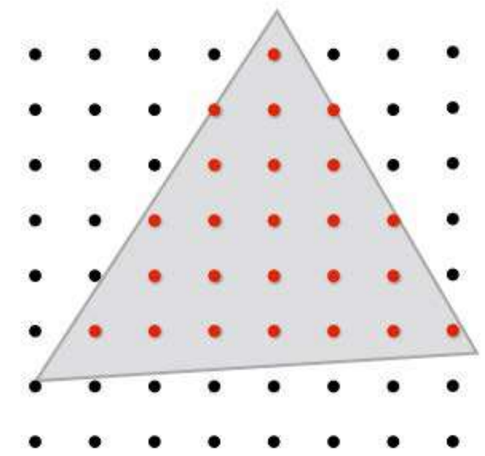
Rough sketch of rasterization pipeline:



Transform/position objects in the world



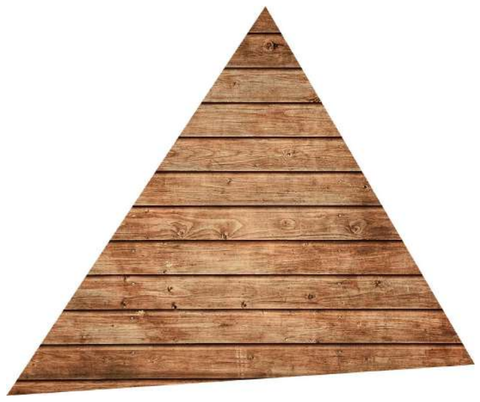
Project objects onto the screen



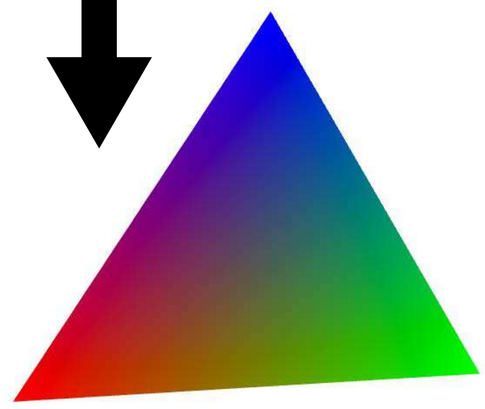
Sample triangle coverage



Combine samples into final image (depth, alpha, …)



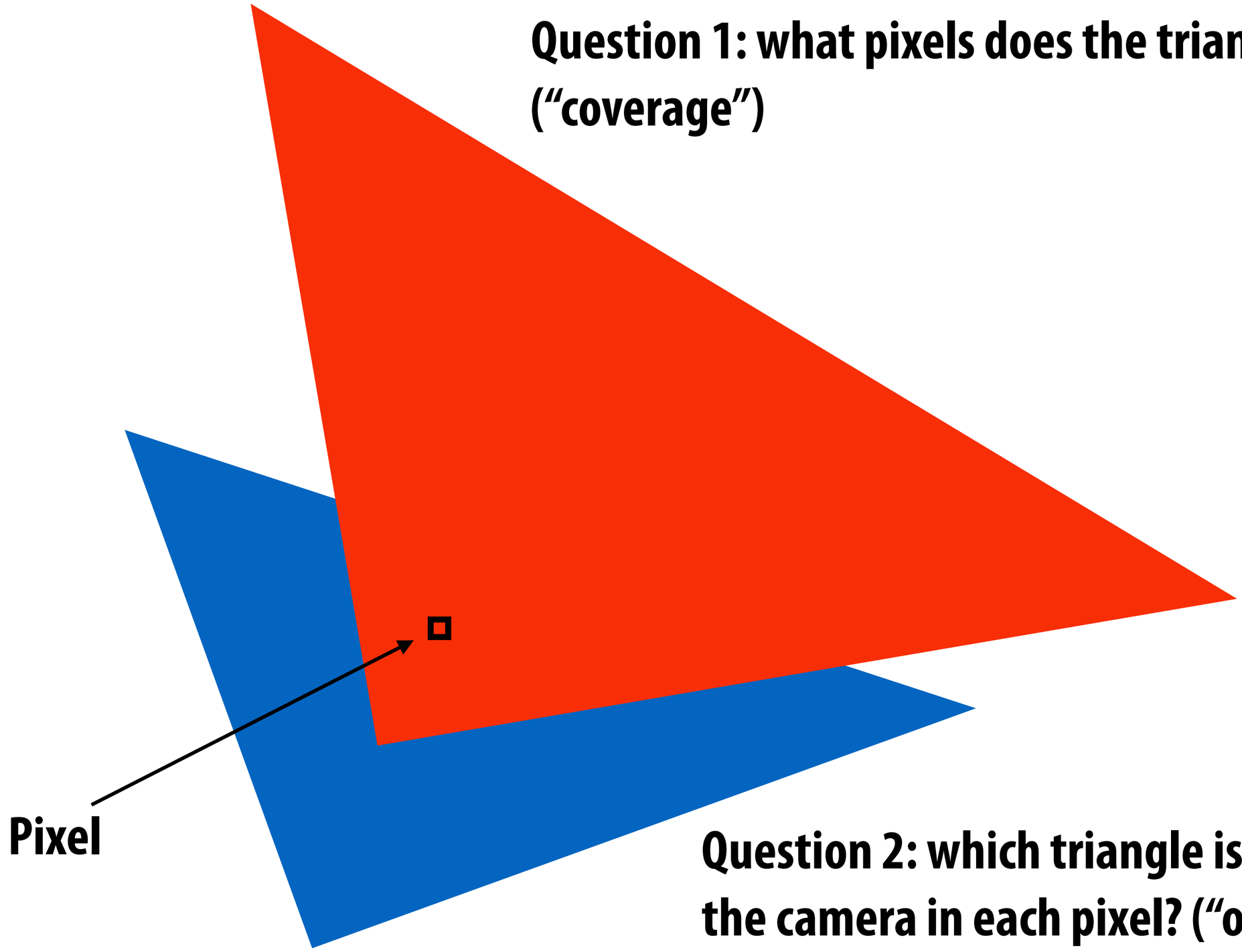
Sample texture maps / evaluate shaders



Interpolate triangle attributes at covered samples

- Reflects standard "real world" pipeline (OpenGL/Direct3D)
  - the rest is just details (e.g., API calls); will discuss in recitation

Let's draw some triangles on the screen

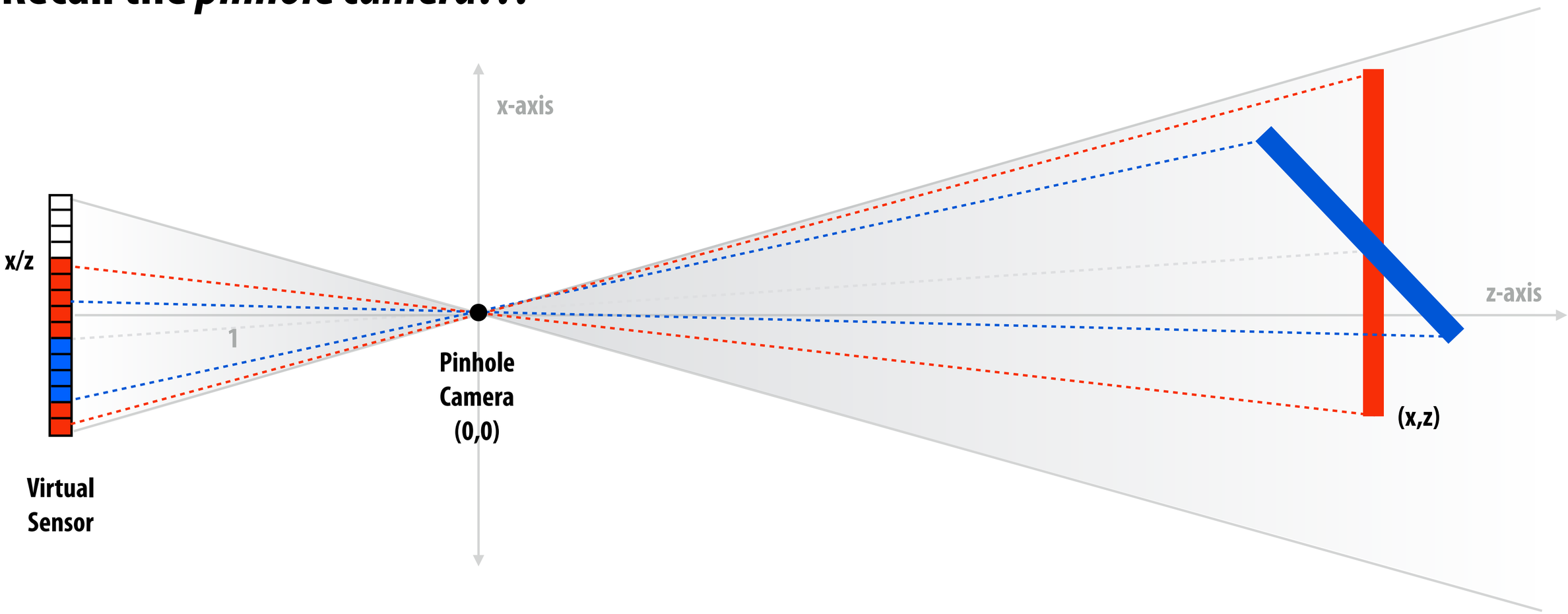


Question 1: what pixels does the triangle overlap? ("coverage")

Question 2: which triangle is closest to the camera in each pixel? ("occlusion")

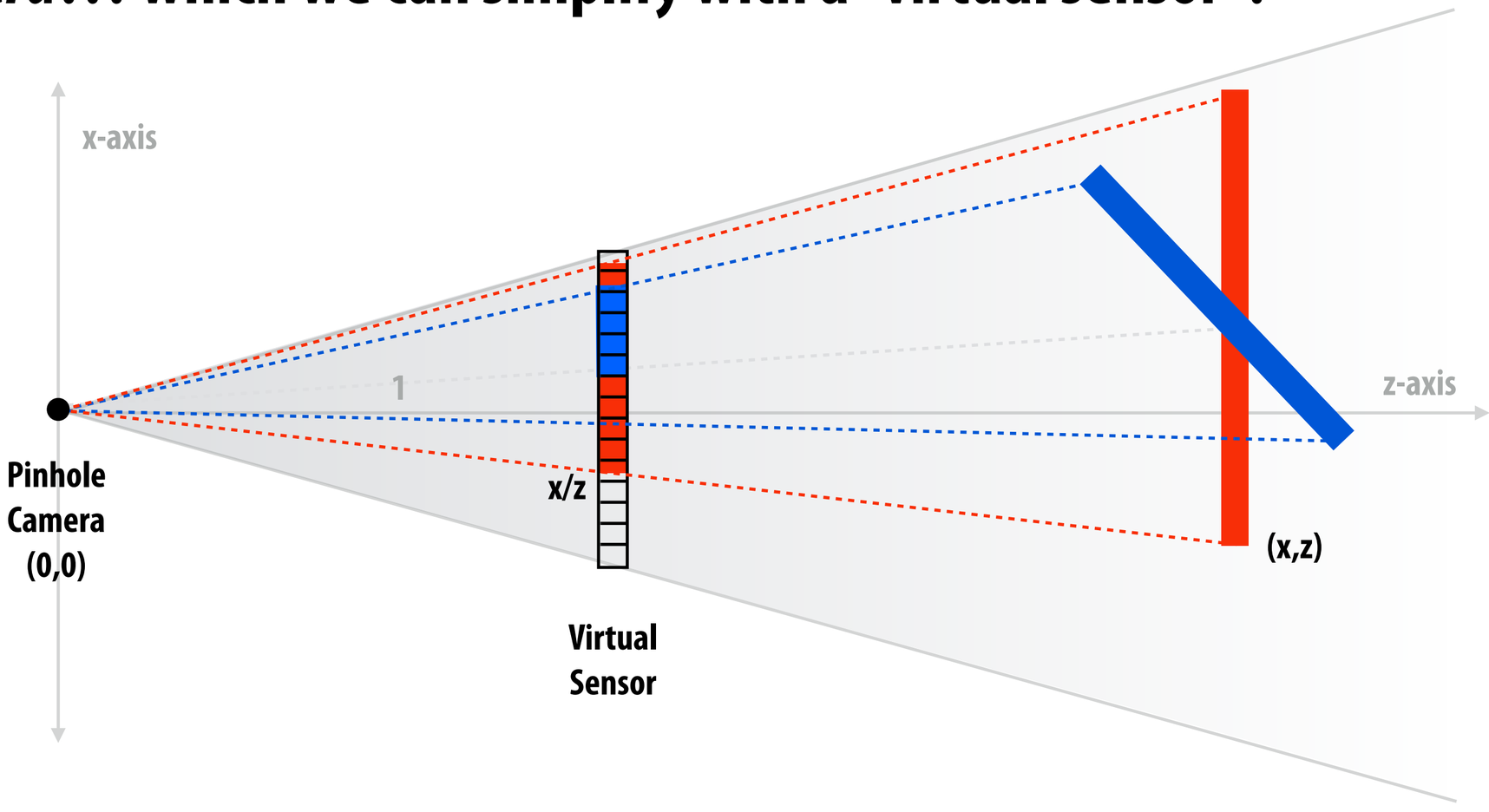
The visibility problem

Recall the *pinhole camera*…



The visibility problem

Recall the *pinhole camera*… which we can simplify with a "virtual sensor":

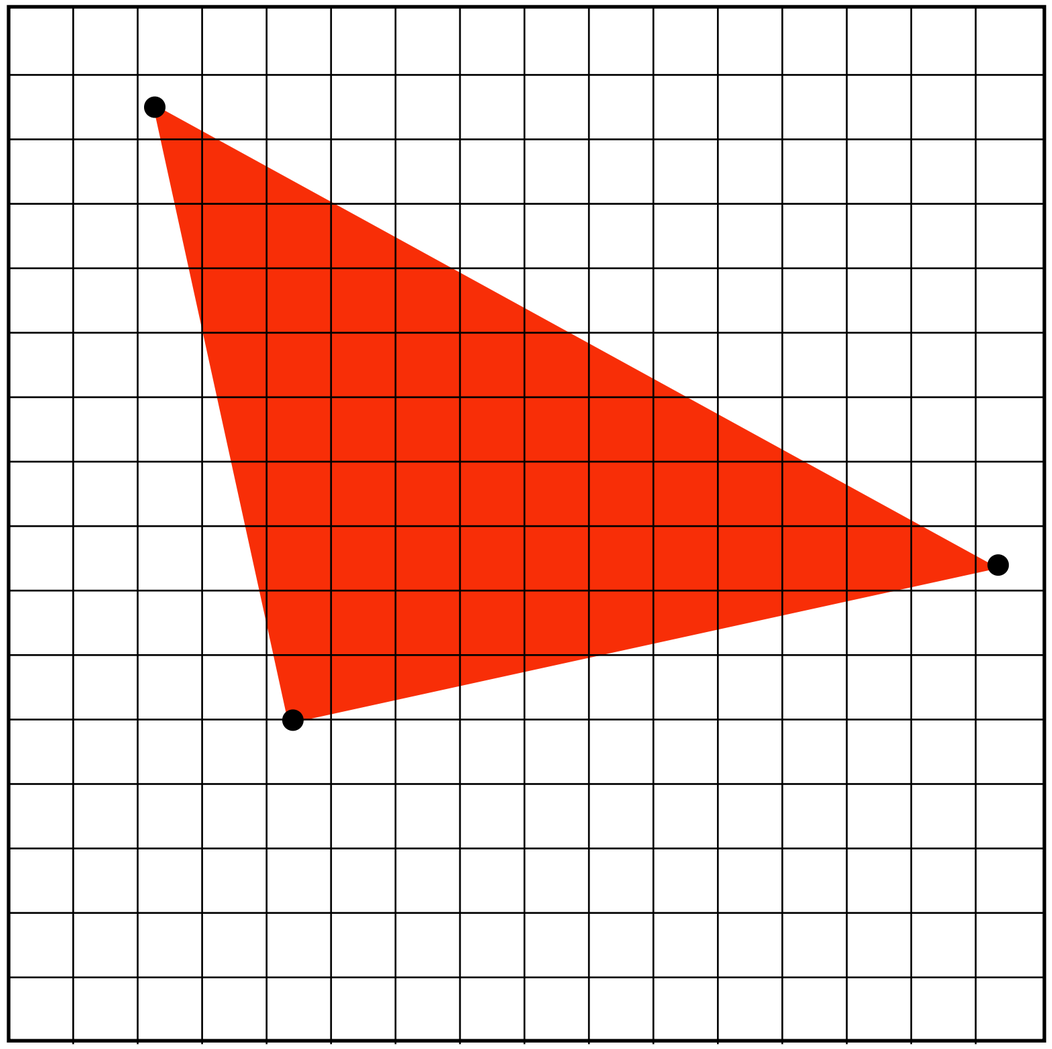


- **Visibility problem in terms of rays:**
  - **COVERAGE: What scene geometry is hit by a ray from a pixel through the pinhole?**
  - **OCCLUSION: Which object is the first hit along that ray?**

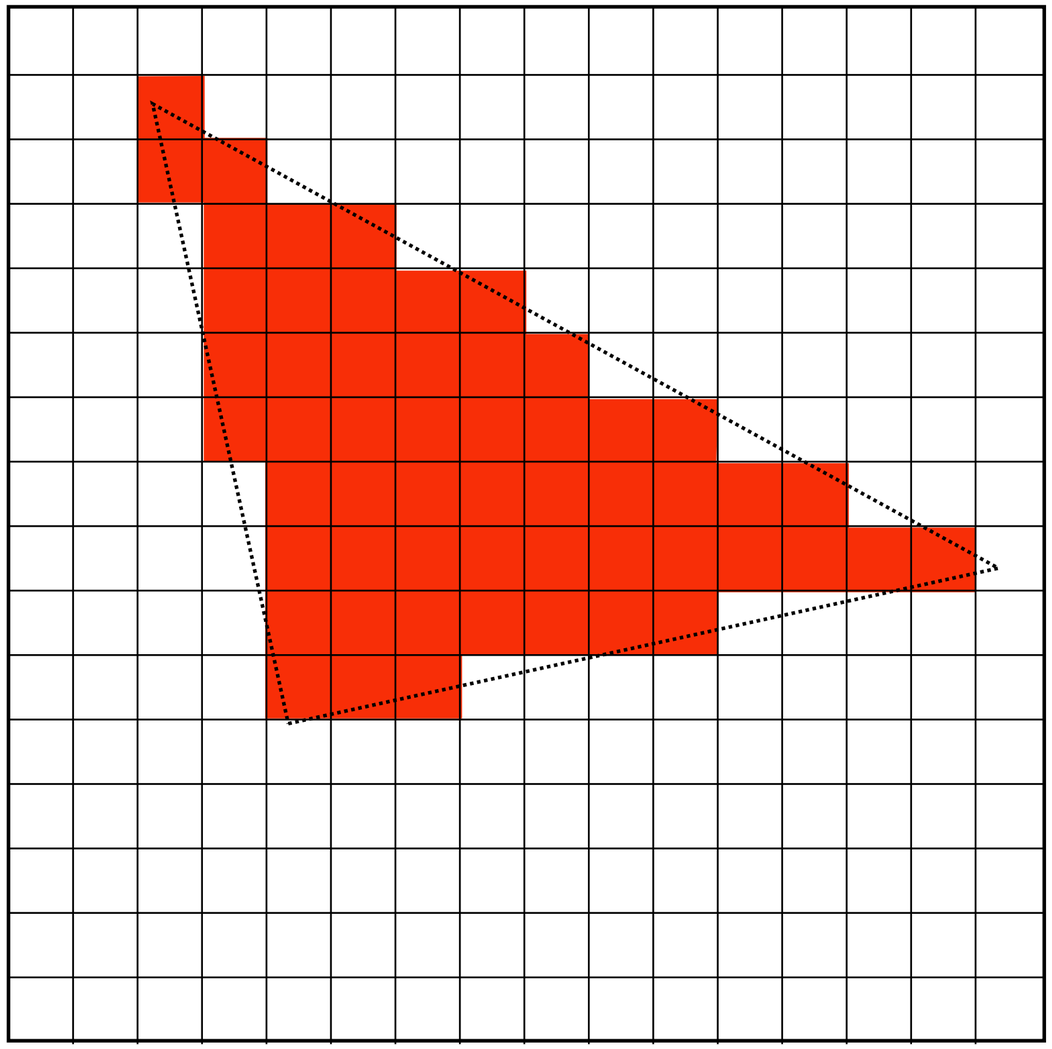
Computing triangle coverage

"Which pixels does the triangle overlap?"

Input:
projected position of triangle vertices: P_0, P_1, P_2

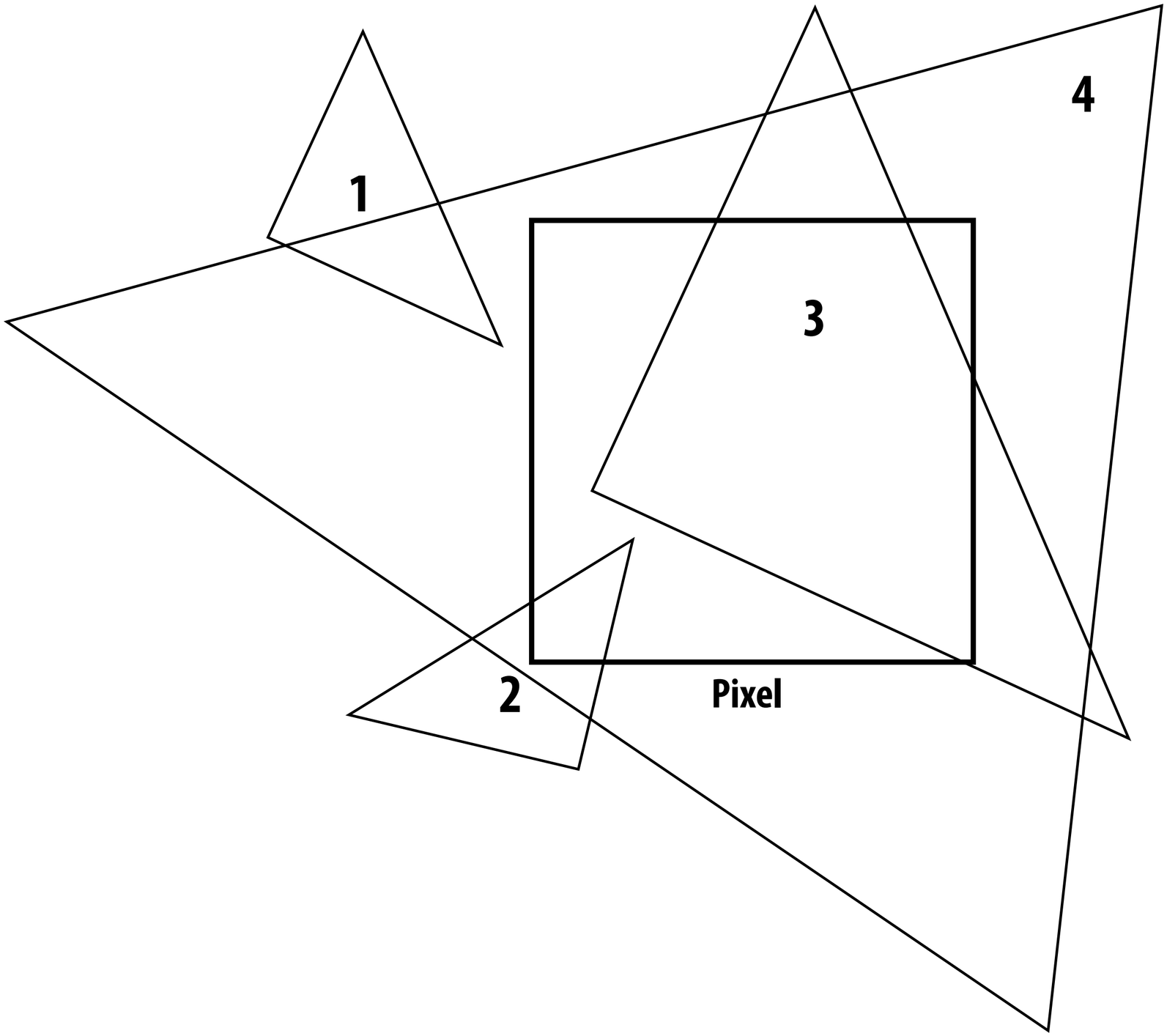


Output:
set of pixels "covered" by the triangle

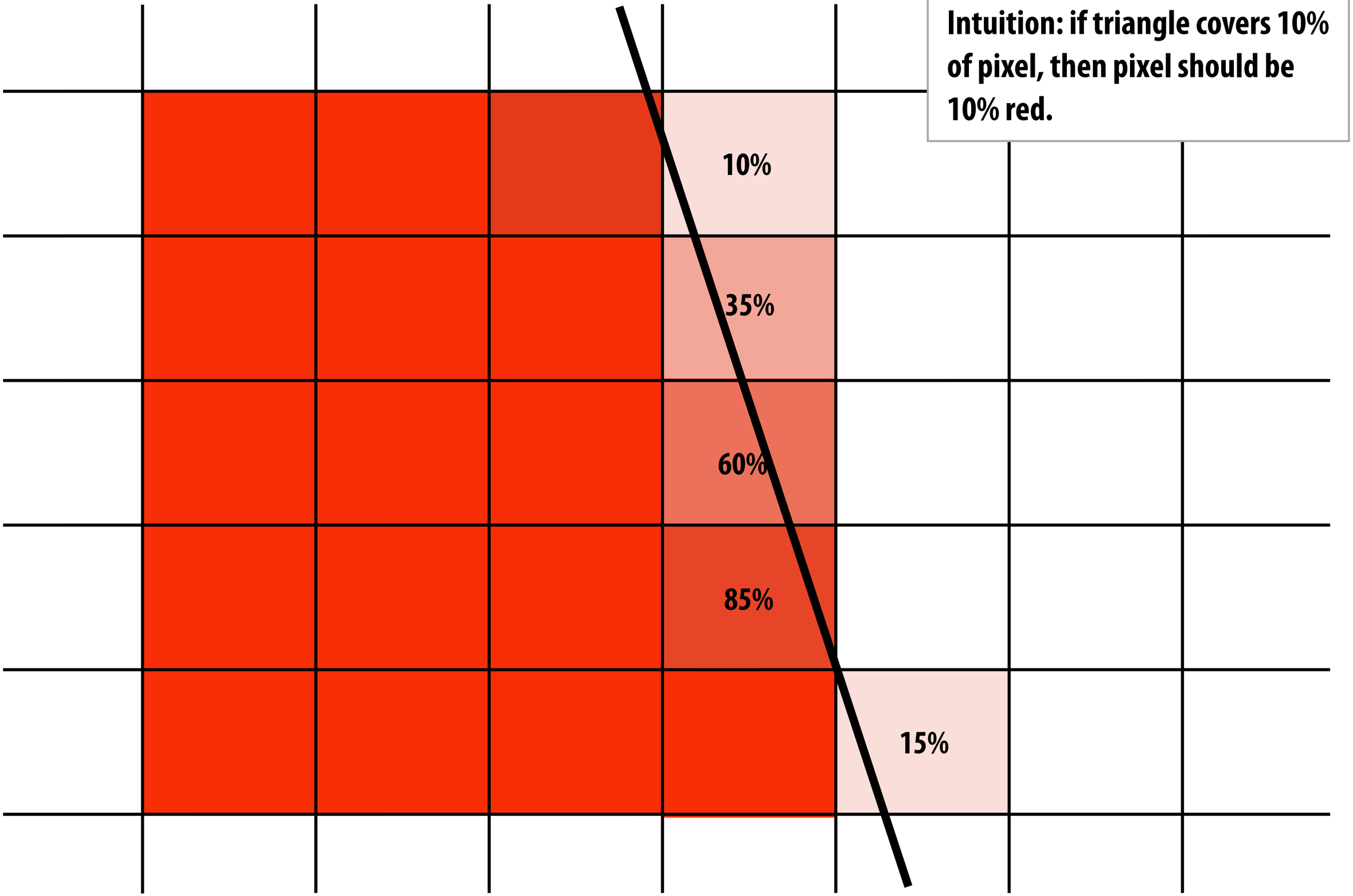


What does it mean for a pixel to be covered by a triangle?

Q: Which triangles "cover" this pixel?

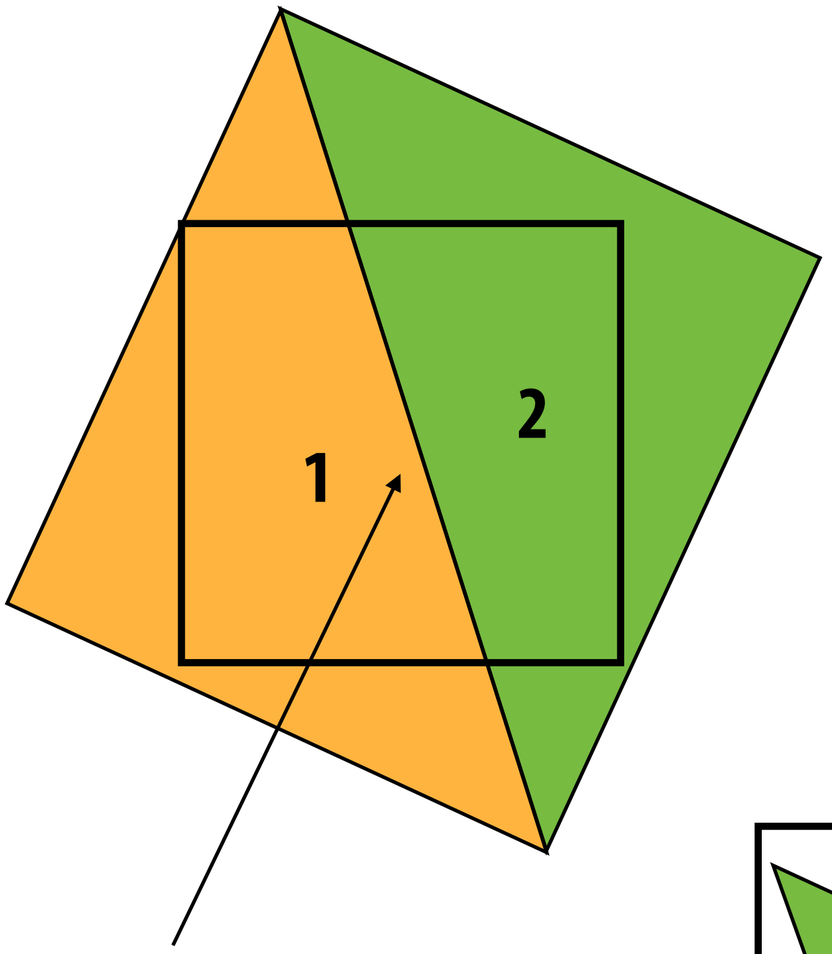


One option: compute fraction of pixel area covered by triangle, then color pixel according to this fraction.

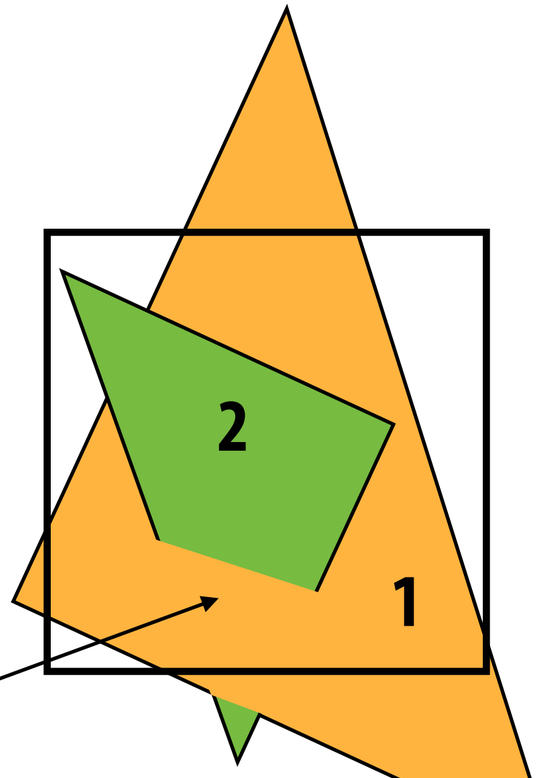


Intuition: if triangle covers 10% of pixel, then pixel should be 10% red.

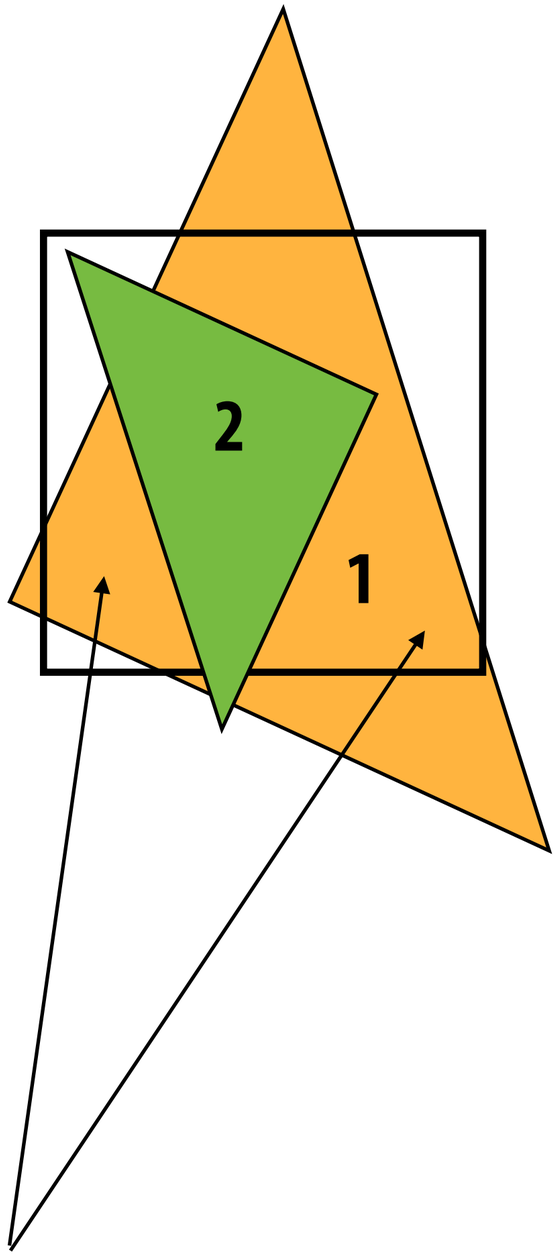
Coverage gets tricky when considering occlusion



Pixel covered by triangle 1, other half covered by triangle 2



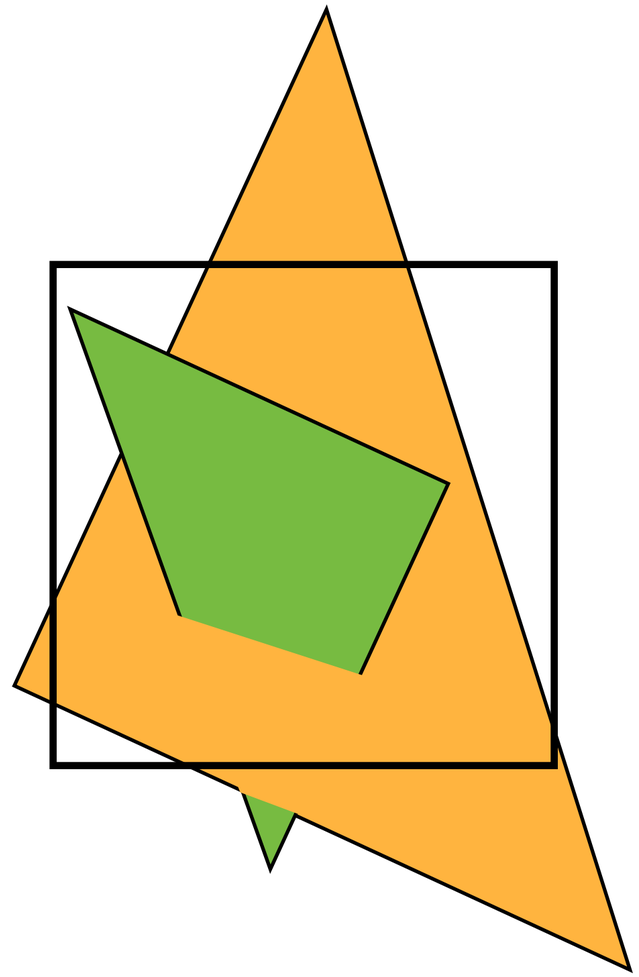
Interpenetration of triangles: even trickier

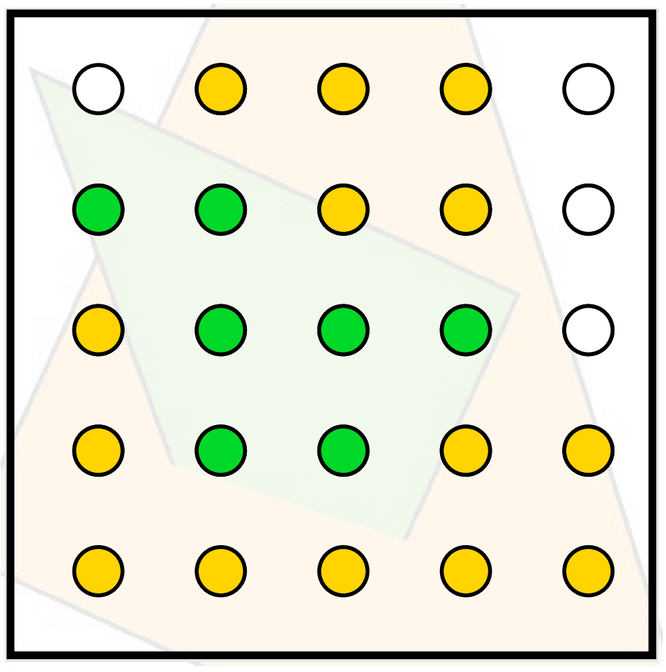


Two regions of triangle 1 contribute to pixel. One of these regions is not even convex.

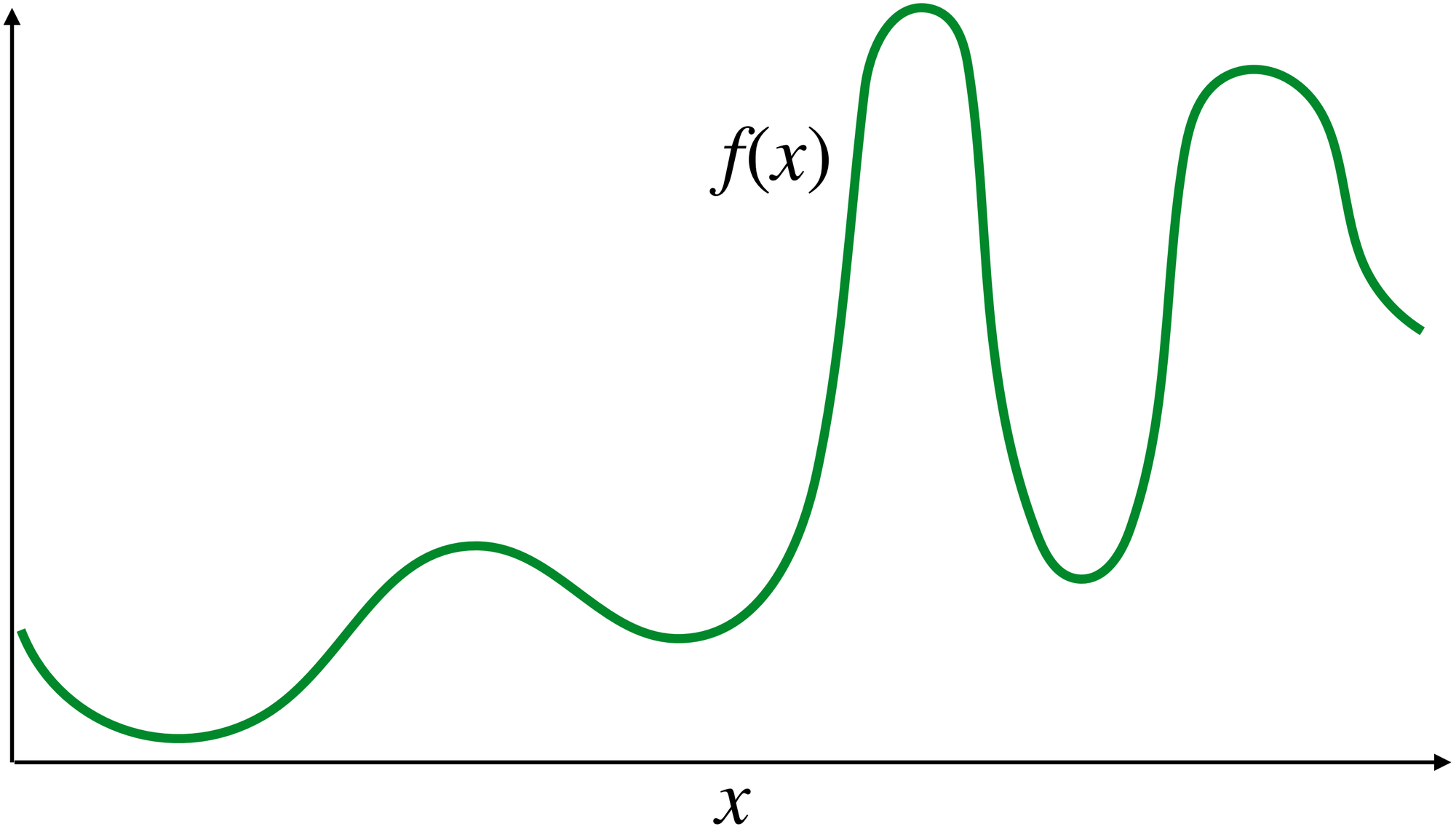
Coverage via sampling

- Real scenes are *complicated!*
  - occlusion, transparency, …
  - will talk about this more in a future lecture!
- Computing *exact* coverage is not practical
- Instead: view coverage as a sampling problem
  - don’t compute exact/analytical answer
  - instead, test a collection of sample points
  - with enough points & smart choice of sample locations, can start to get a good estimate
- First, let’s talk about sampling in general…



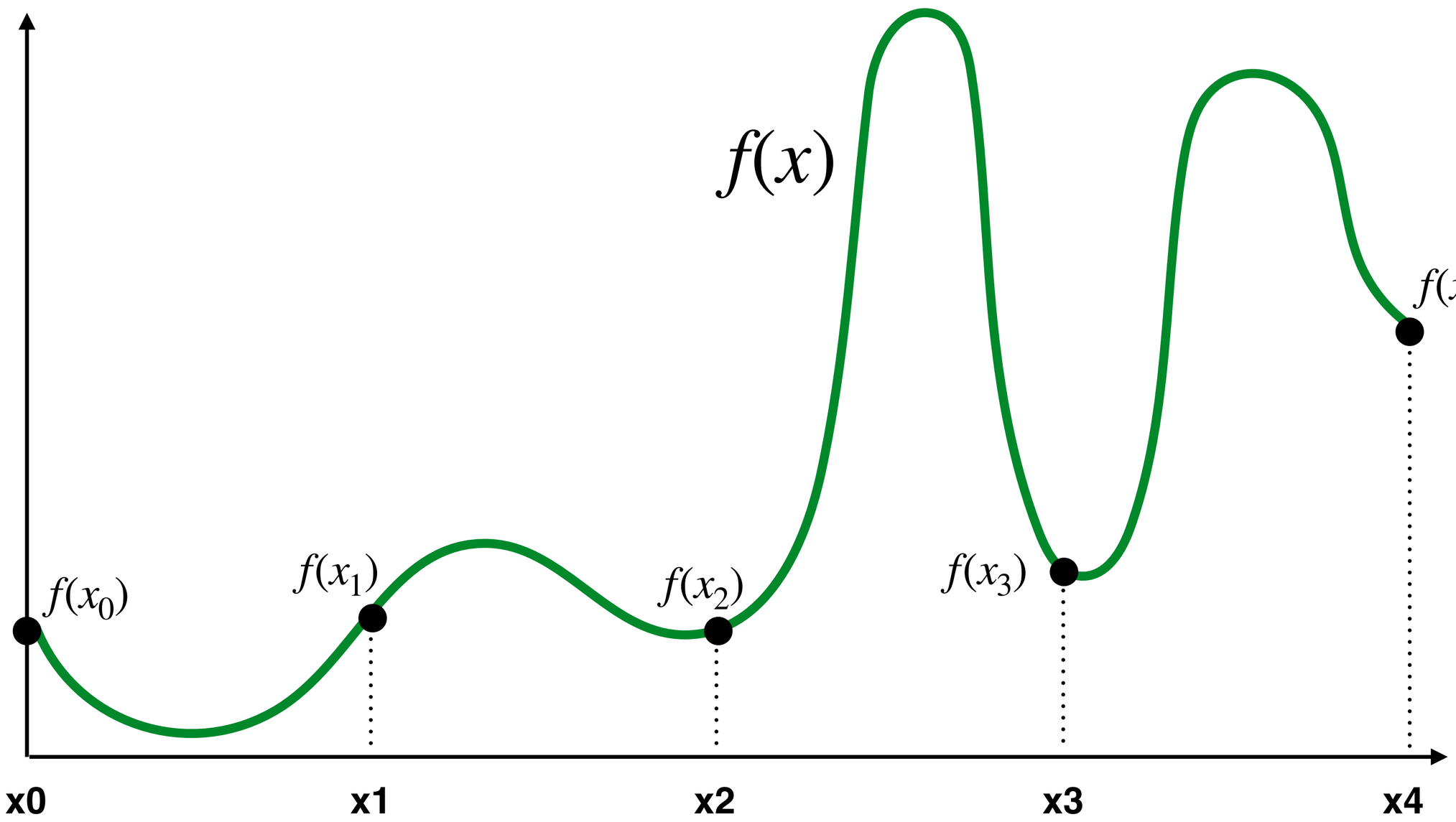


Sampling 101: Sampling a 1D signal

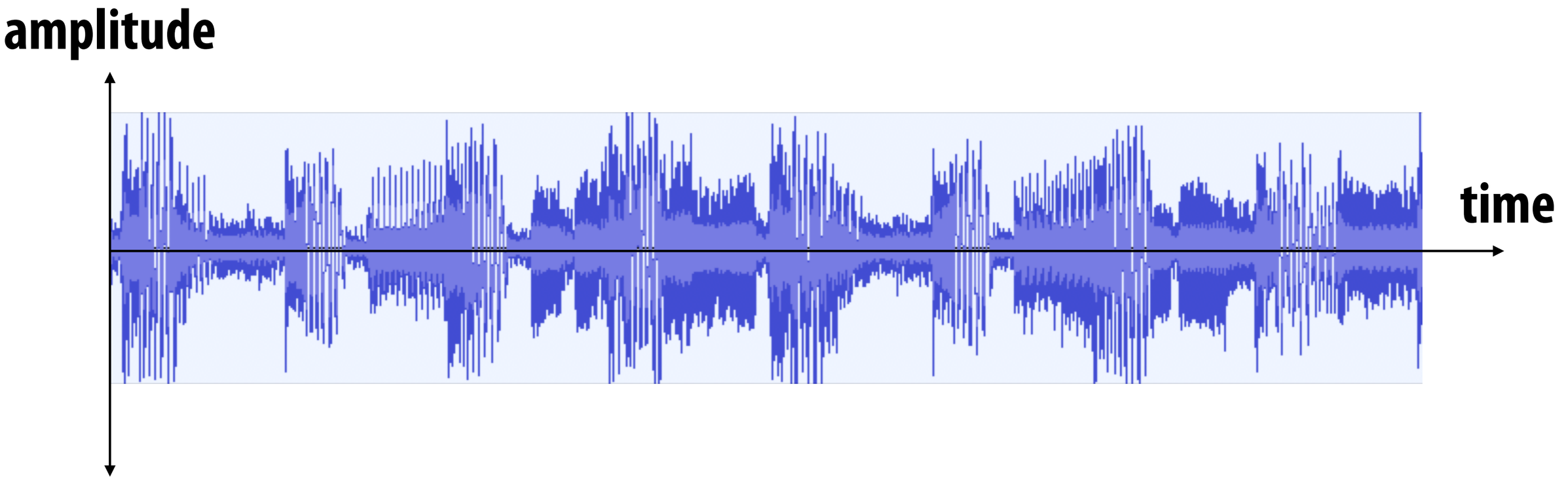


Sampling = taking measurements of a signal

Below: 5 measurements ("samples") of $f(x)$

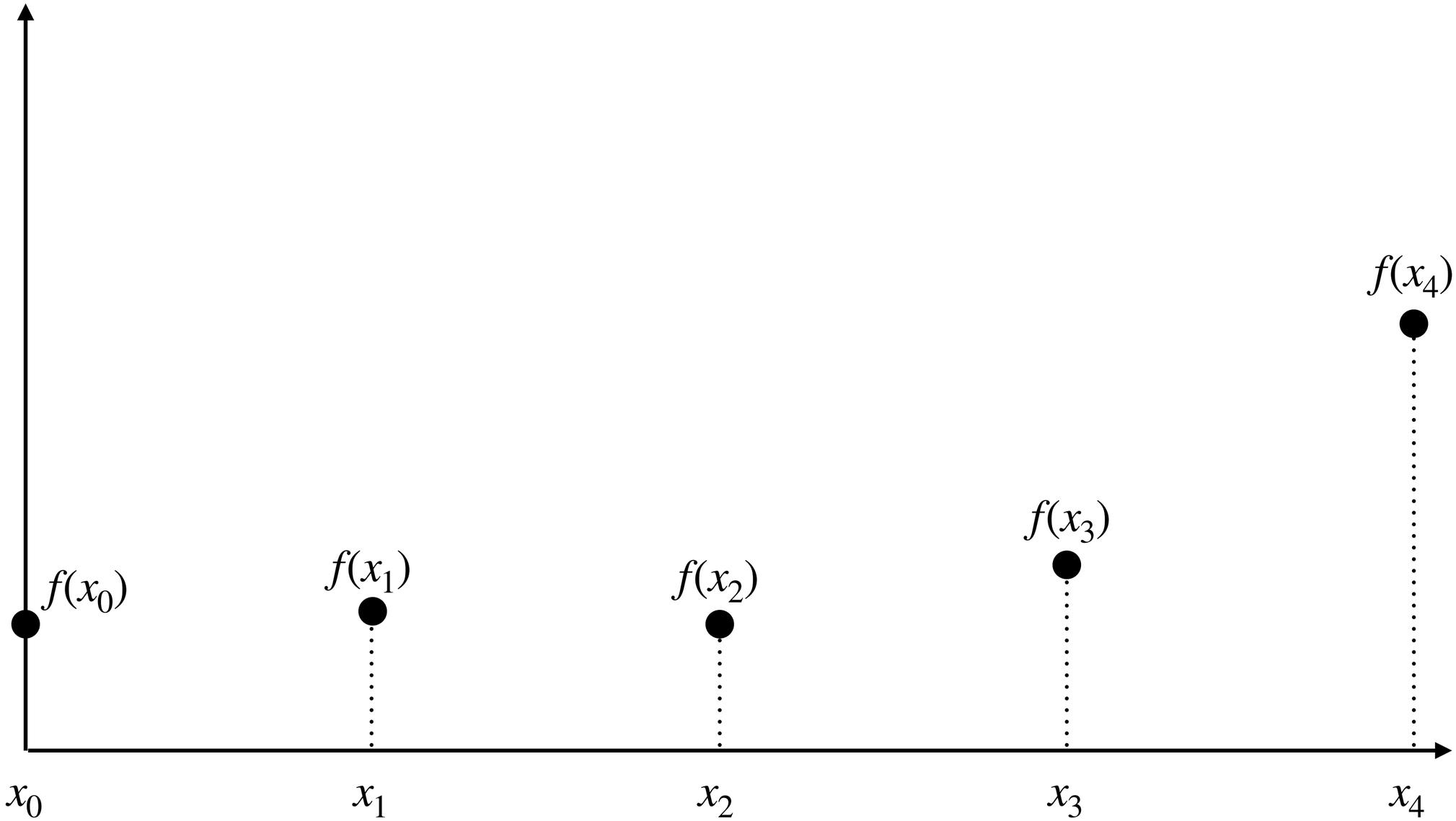


Audio file: stores samples of a 1D signal



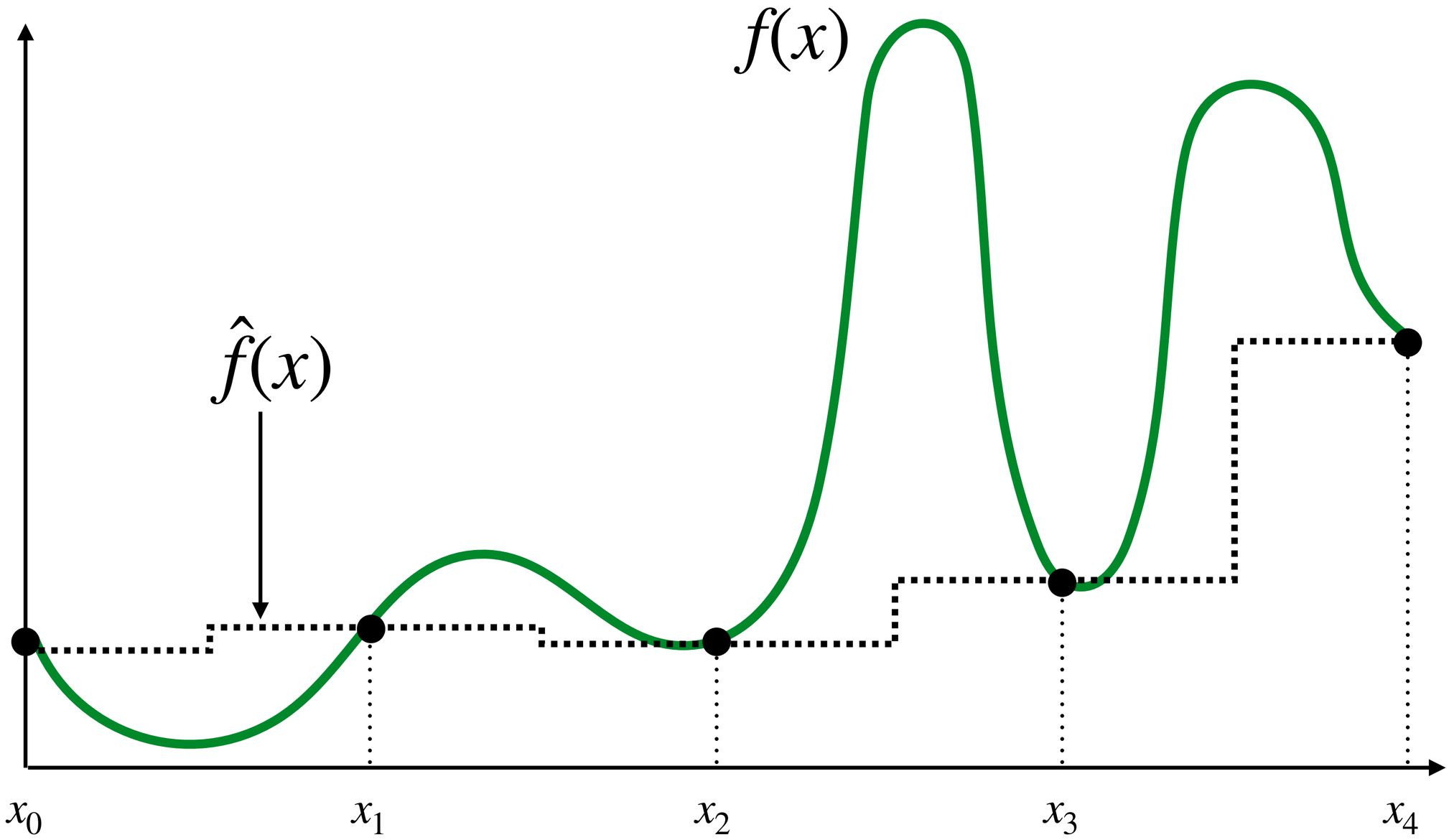
(most consumer audio is sampled 44,100 times per second, i.e., at *44.1 KHz*)

Reconstruction: given a set of samples, how might we attempt to reconstruct the original signal $f(x)$?



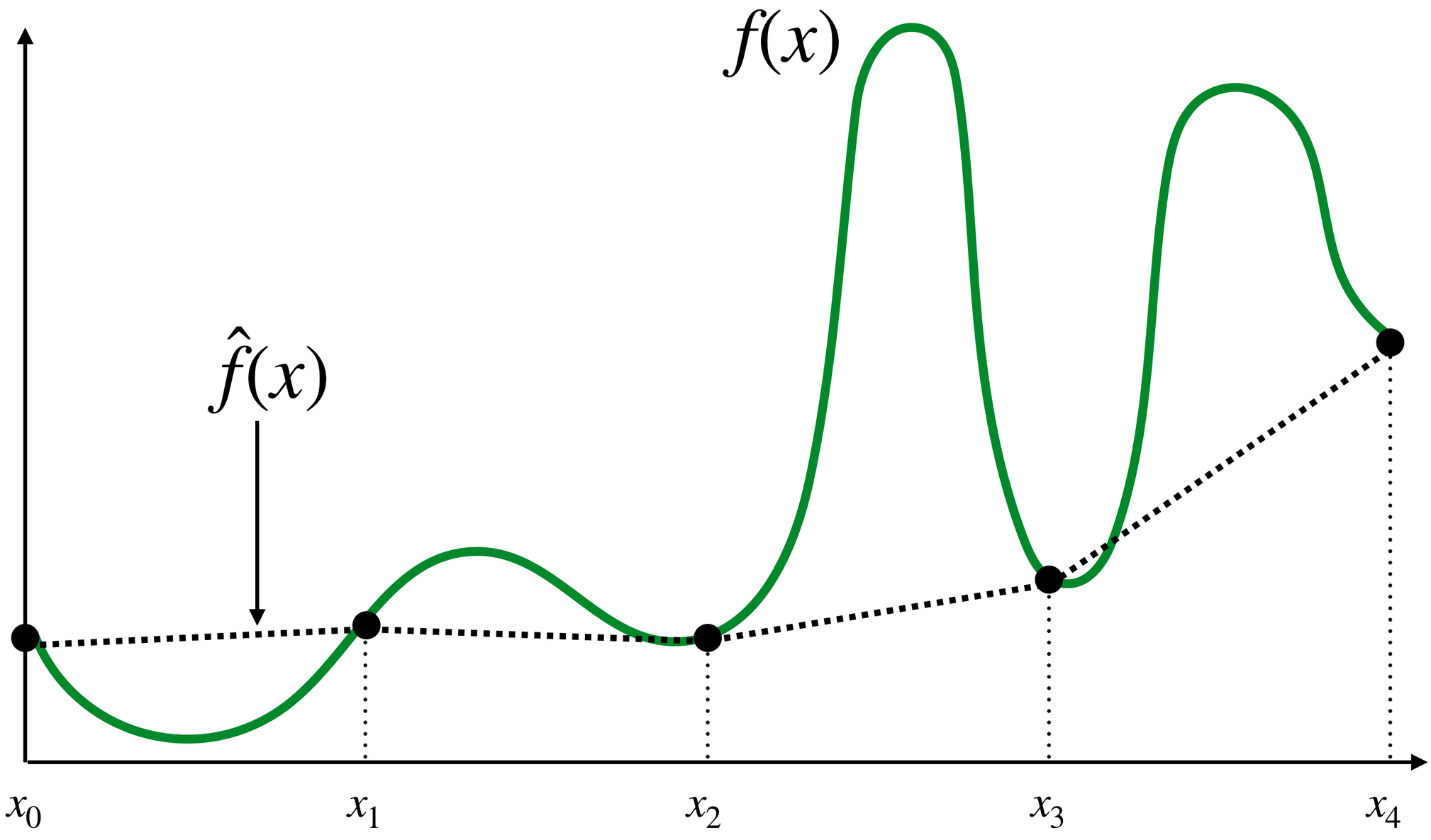
Piecewise constant approximation

$\hat{f}(x)$ = value of sample closest to x



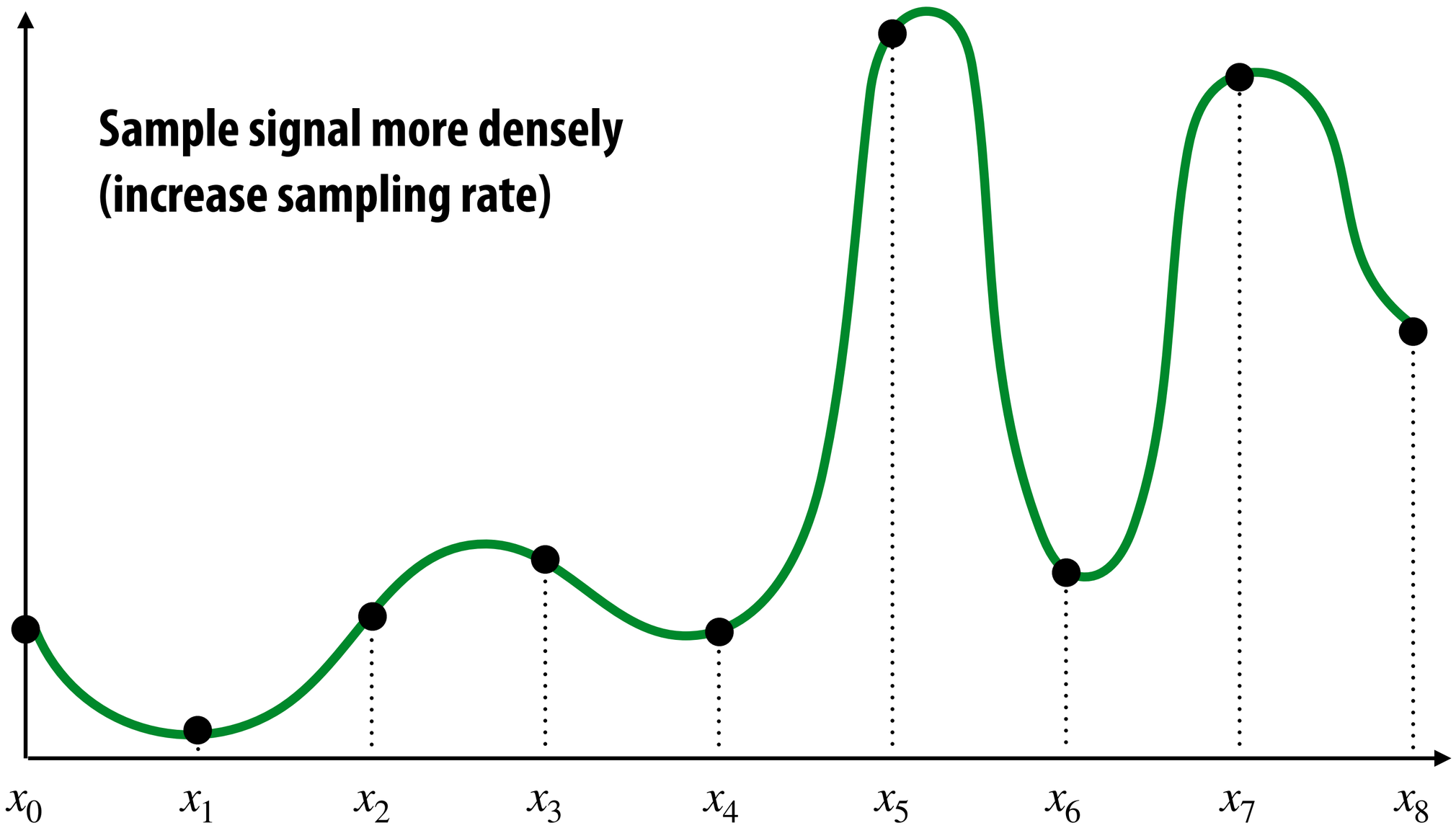
Piecewise linear approximation

$\hat{f}(x)$ = linear interpolation between values of two closest samples to x

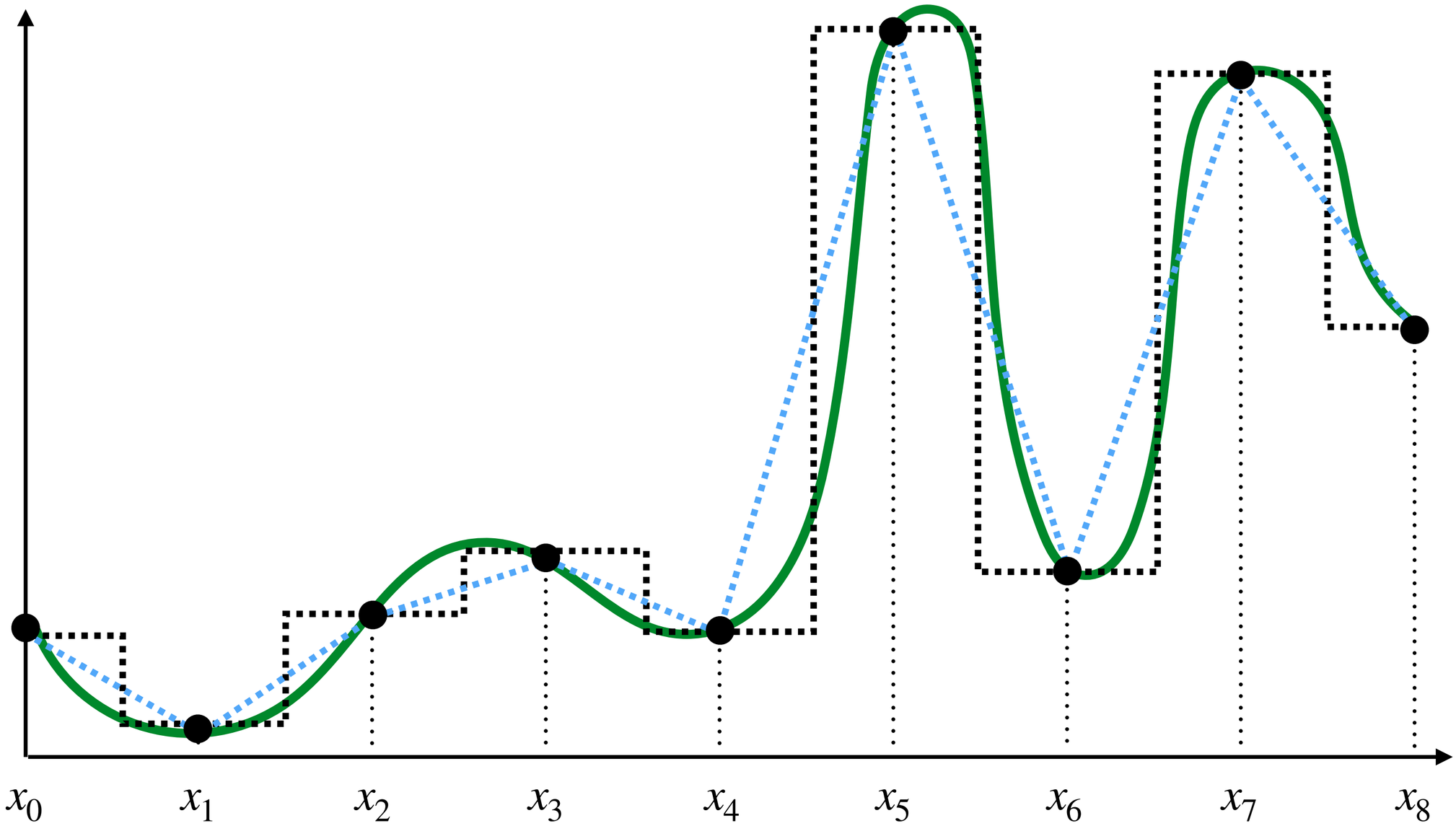


How can we represent the signal more accurately?

Sample signal more densely
(increase sampling rate)



Reconstruction from denser sampling



= reconstruction via nearest

= reconstruction via linear interpolation

2D Sampling & Reconstruction

- Basic story doesn't change much for images:
  - sample values measure image (i.e., signal) at sample points
  - apply interpolation/reconstruction filter to approximate image



original



piecewise constant ("nearest neighbor")

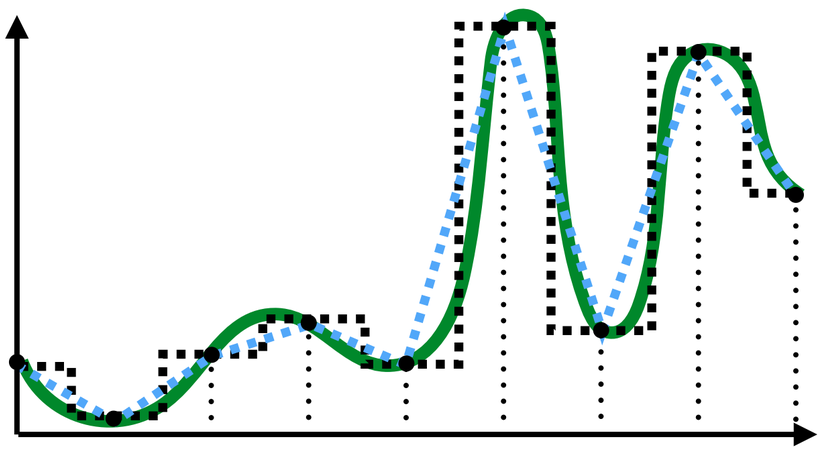


piecewise *bi*-linear

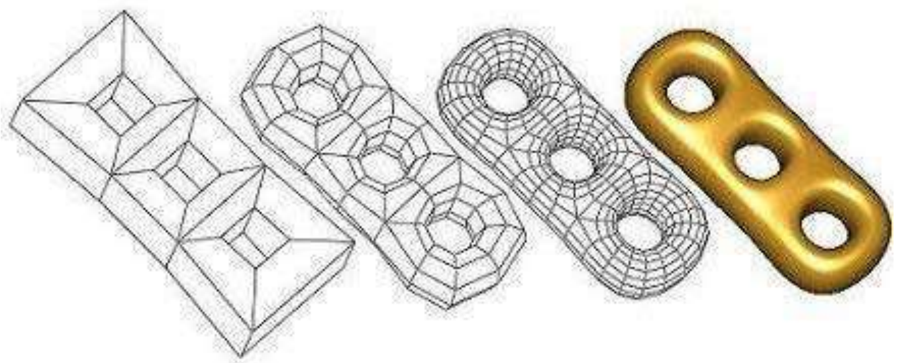
Sampling 101: Summary

- Sampling = measurement of a signal
  - Encode signal as discrete set of samples
  - In principle, represent values at specific points (though hard to measure in reality!)

- Reconstruction = generating signal from a discrete set of samples
  - Construct a function that interpolates or approximates function values
  - E.g., piecewise constant/"nearest neighbor", or piecewise linear
  - Many more possibilities! For all kinds of signals (audio, images, geometry…)



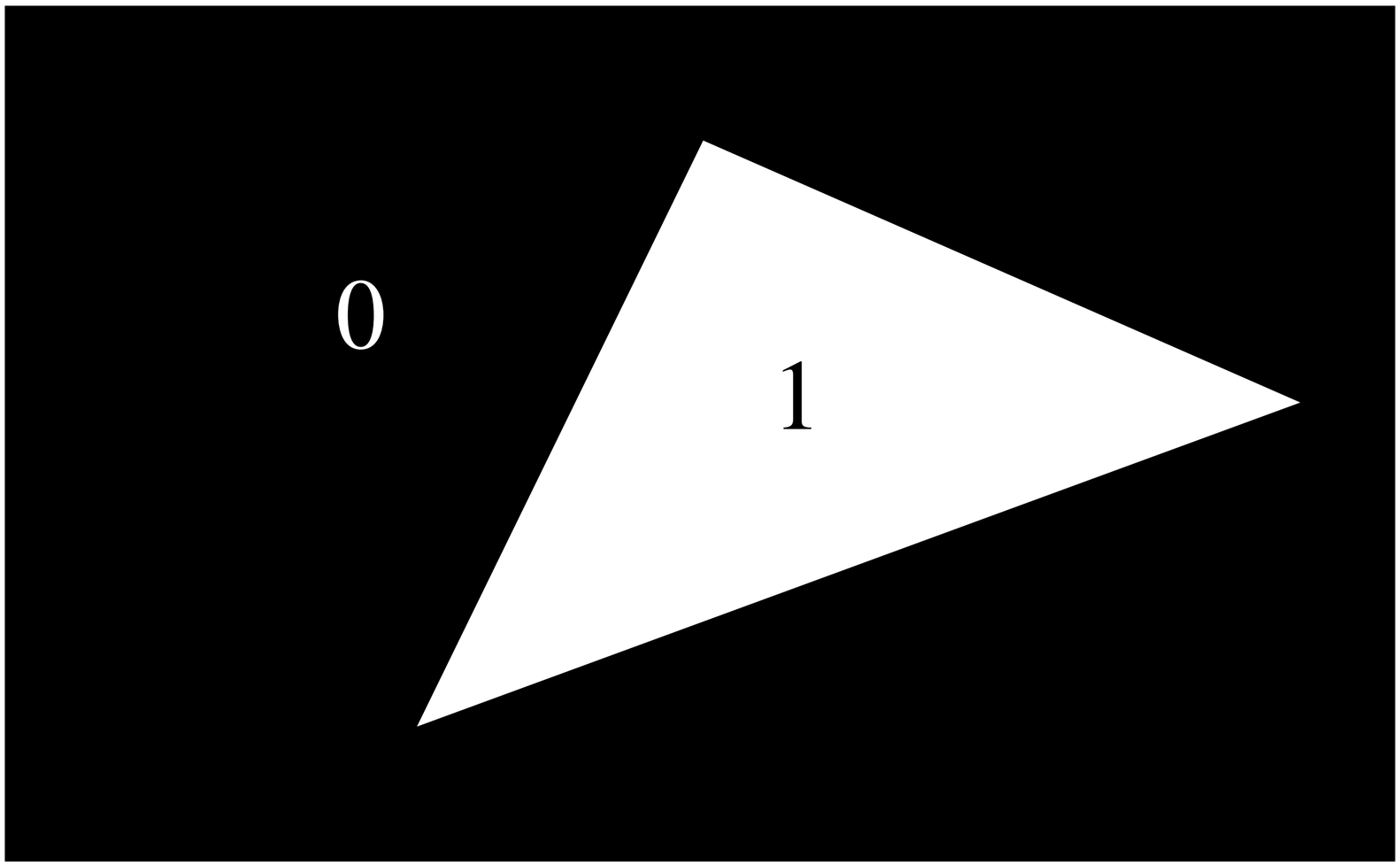




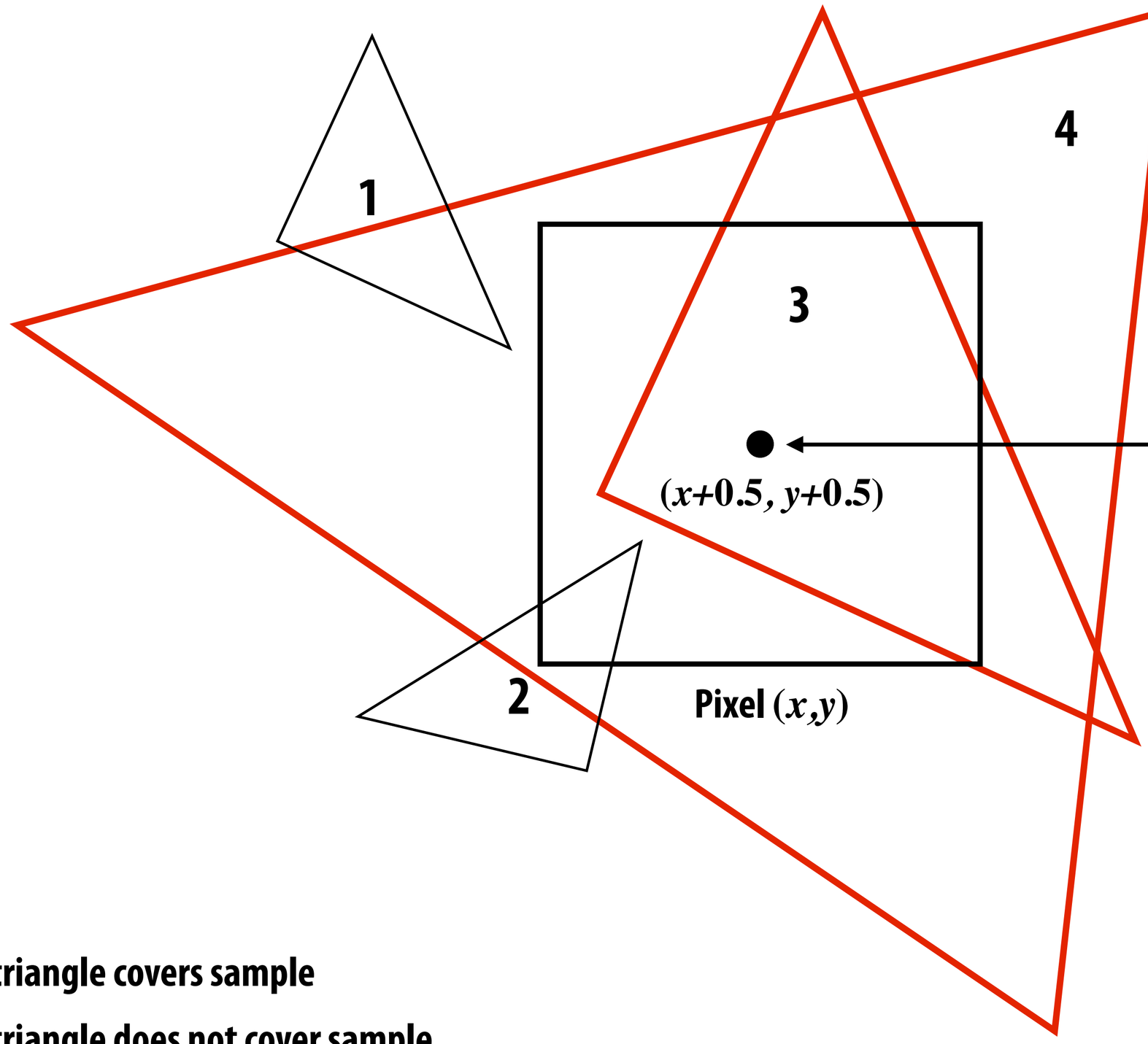
[Image credit: Wikipedia]

For rasterization, what function are we sampling?

$$\text{coverage}(x, y) := \begin{cases} 1, & \text{triangle contains point } (x, y) \\ 0, & \text{otherwise} \end{cases}$$



Simple rasterization: just sample the coverage function



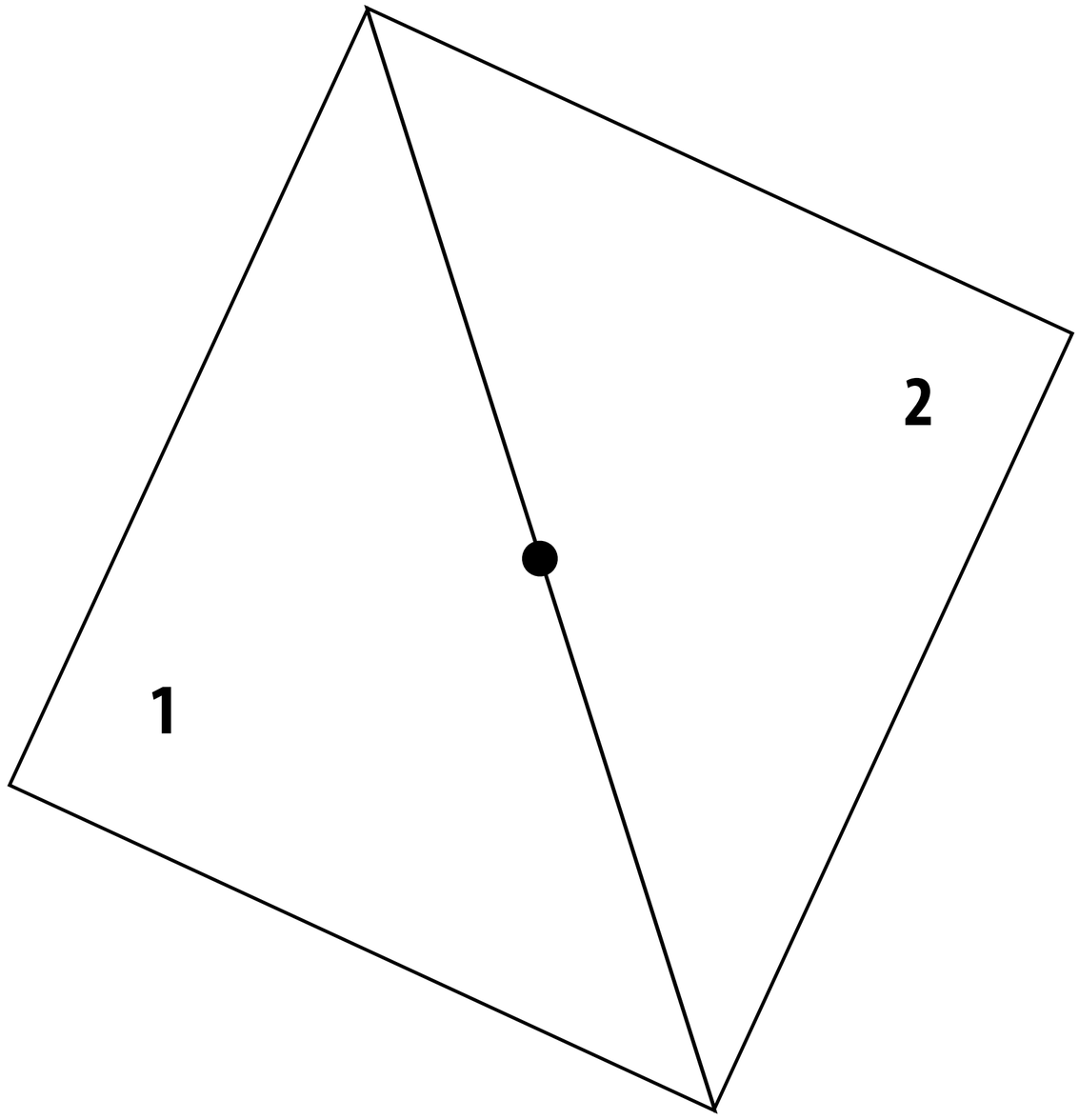
Example:
Here I chose the coverage sample point to be at a point corresponding to the pixel center.

= triangle covers sample

= triangle does not cover sample

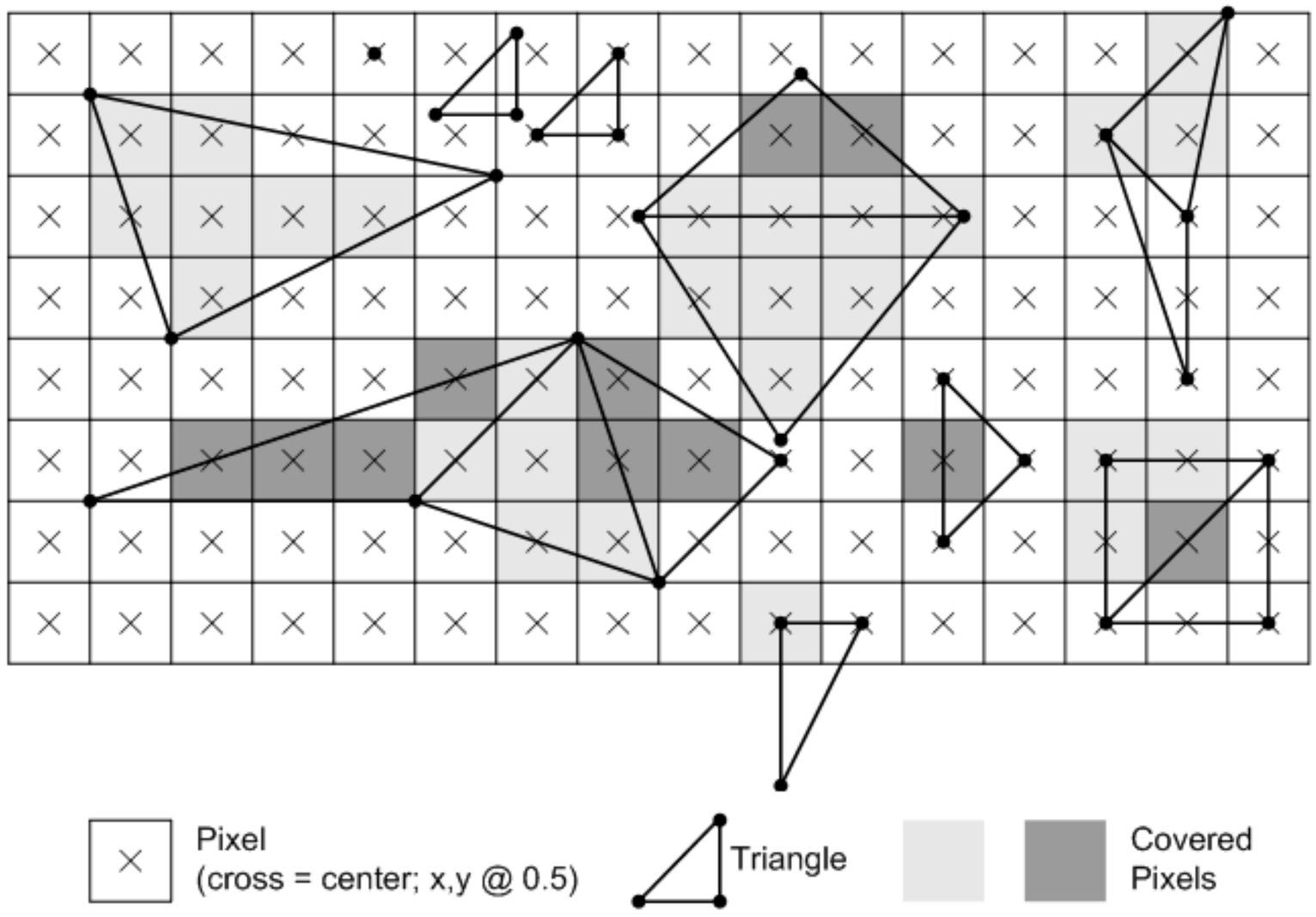
Edge cases (literally)

Is this sample point covered by triangle 1? or triangle 2? or both?



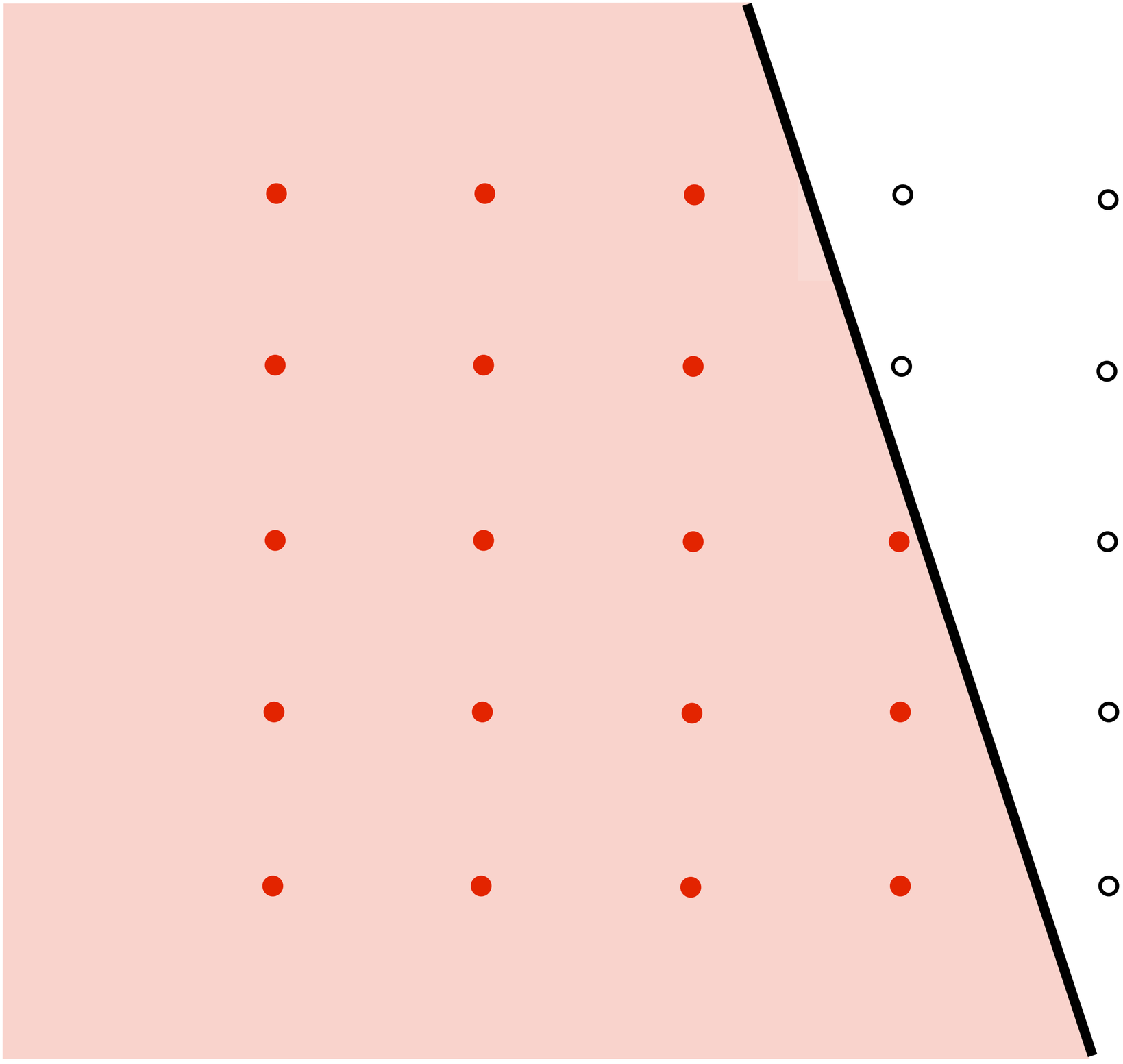
Breaking Ties*

- When edge falls directly on a screen sample point, the sample is classified as within triangle if the edge is a "top edge" or "left edge"
  - Top edge: horizontal edge that is above all other edges
  - Left edge: an edge that is not exactly horizontal and is on the left side of the triangle. (triangle can have one or two left edges)

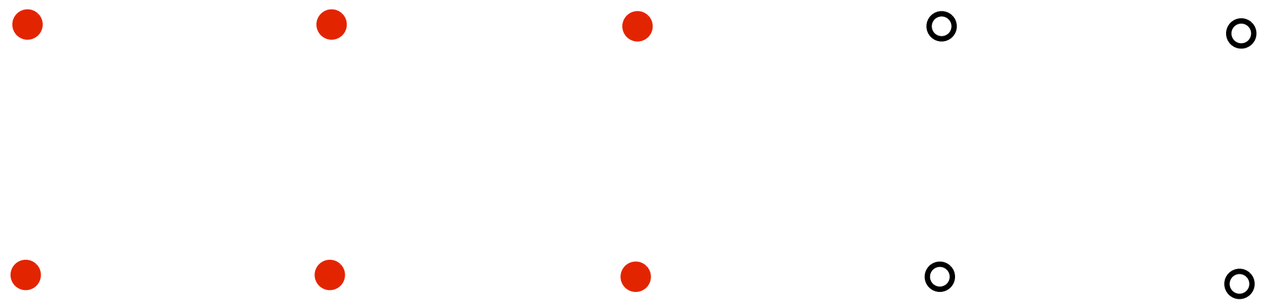


*These are the rules used in OpenGL/Direct3D, i.e., in modern GPUs. Source: Direct3D Programming Guide, Microsoft

Results of sampling triangle coverage

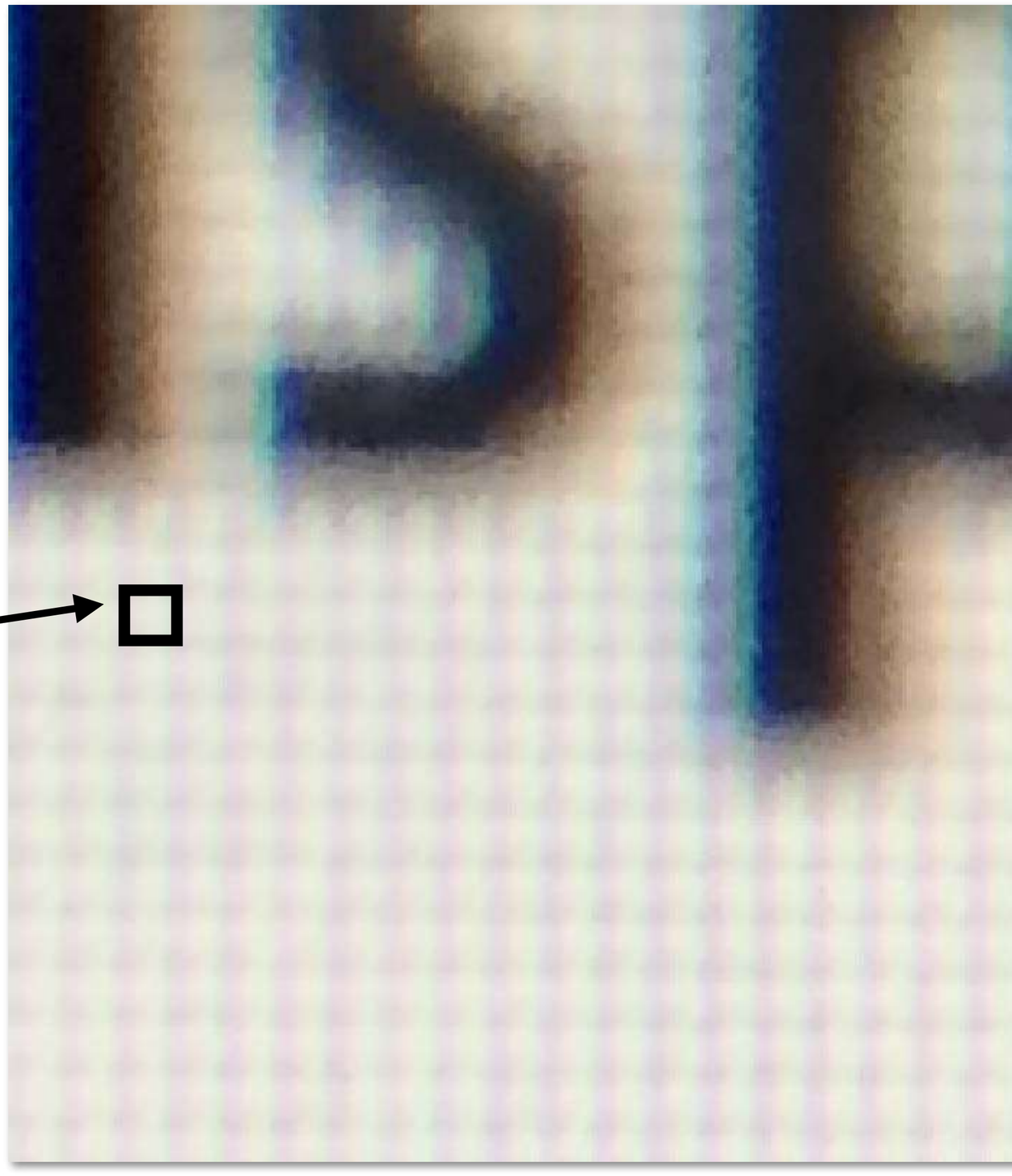


I have a sampled signal, now I want to display it on a screen



Pixels on a screen

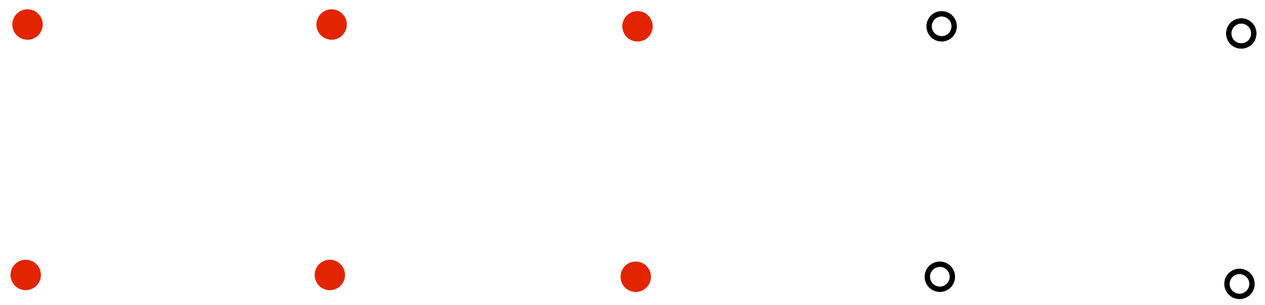
Each image sample sent to the display is converted into a little square of light of the appropriate color:
(a pixel = picture element)



LCD display pixel on my laptop

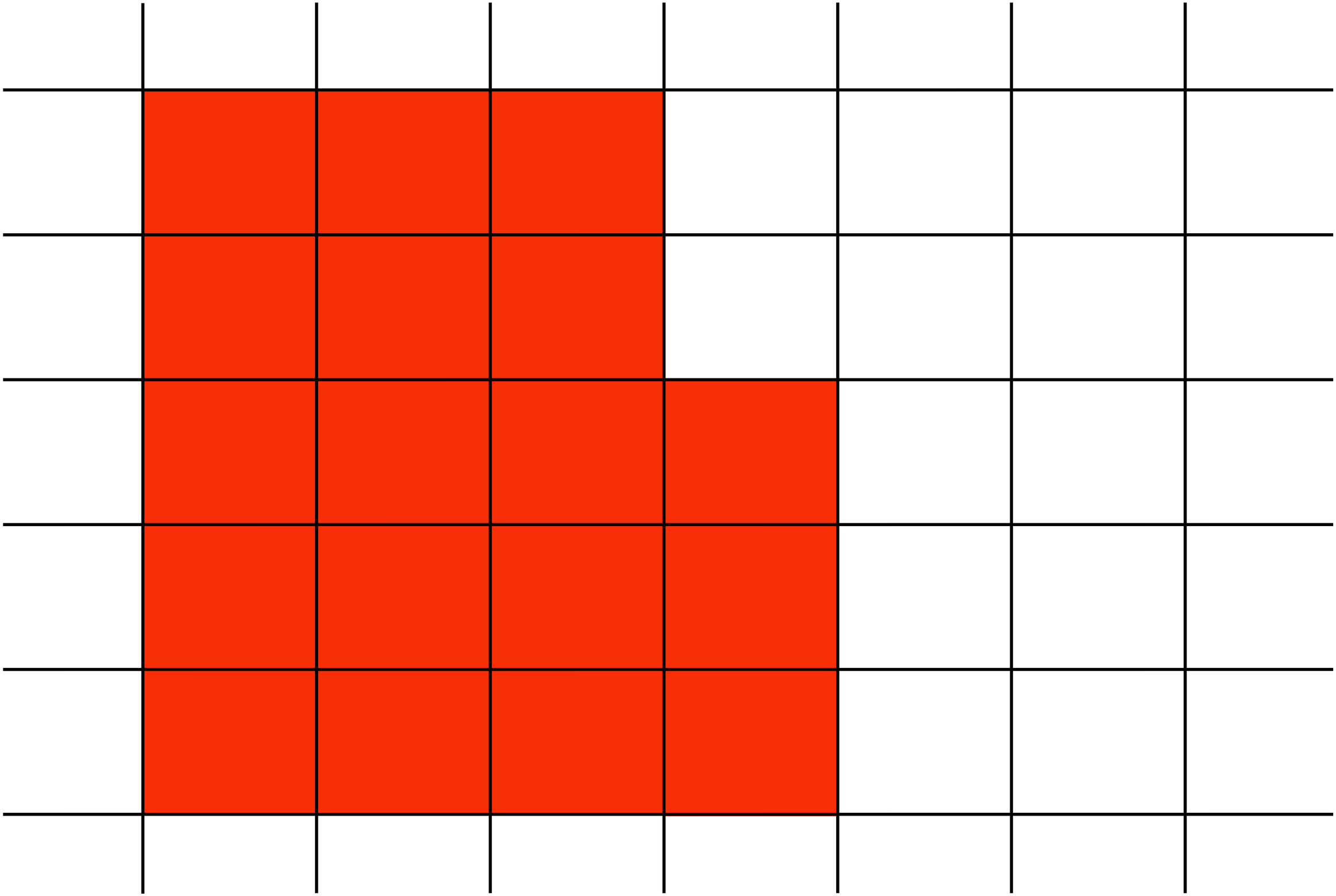
*** Thinking of each LCD pixel as emitting a square of uniform intensity light of a single color is a bit of an approximation to how real displays work, but it will do for now.**

So if we send the display this:



We see this when we look at the screen

(assuming a screen pixel emits a square of perfectly uniform intensity of light)



But the real coverage signal looked like this!

